

北京邮电大学

硕士学位论文



题目： 基于 DPDK 的 IPv4/IPv6 翻译器
的设计与实现

学 号： 2012111421

姓 名： 张宇烜

专 业： 软件工程

导 师： 马严

学 院： 网络技术研究院

2014 年 12 月 24 日

中国 · 北京

北京邮电大学

硕士学位论文



题目： 基于 DPDK 的 IPv4/IPv6 翻译器
的设计和实现

学 号： 2012111421

姓 名： 张宇烜

专 业： 软件工程

导 师： 马严

学 院： 网络技术研究院

2014 年 12 月 24 日

独创性（或创新性）声明

本人声明所呈交的论文是本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京邮电大学或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

申请学位论文与资料若有不实之处，本人承担一切相关责任。

本人签名： 张宇旭

日期： 2015.3.17

关于论文使用授权的说明

学位论文作者完全了解北京邮电大学有关保留和使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权单位属北京邮电大学。学校有权保留并向国家有关部门或机构送交论文的复印件和磁盘，允许学位论文被查阅和借阅；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。（保密的学位论文在解密后遵守此规定）

保密论文注释：本学位论文属于保密在 年解密后适用本授权书。非保密论文注释：本学位论文不属于保密范围，适用本授权书。

本人签名： 张宇旭

日期： 2015.3.17

导师签名： 王

日期： 2015.3.17

基于 DPDK 的 IPv4/IPv6 翻译器的设计和实现

摘 要

随着 IPv4 协议地址的数目枯竭和 IPv6 技术的日益成熟, IPv6 商用化已经迫在眉睫。但是由于 IPv4 地址格式与 IPv6 地址格式并不兼容, 现有的设备不能从 IPv4 直接升级到 IPv6。IPv4 网络与 IPv6 网络不能直接访问。在这种背景下, IPv4/IPv6 过渡技术就显得尤为重要, 通过 IPv4/IPv6 过渡技术, 可以在用户无感知的情况下, 完成 IPv4 网络到 IPv6 网络的平滑过渡。

同时, 现阶段由于整个全球互联网的流量不断增加, 服务器并发连接的带宽也提升到每秒千兆的级别。服务器的性能受到了极大的制约。目前单纯依靠硬件的升级的途径已经无法从根本上解决超大流量的并发。人们开始从服务器实现, 数据库的组织方式甚至系统内核工作方式等软件方面进行改进, 以提高整个系统的处理网络并发流量的能力。

本文通过研究有状态的地址翻译技术和高性能的数据流量并行技术, 在 Intel 的 DPDK (Data Plane Development Kit) 高性能数据平台上实现了一个简单的 IPv4 和 IPv6 数据报文转发平台和有状态的 IPv4/IPv6 地址翻译器。通过二者的结合, 设计了一个基于 DPDK 的 IPv4/IPv6 翻译器。IP 数据报文转发平台通过使用 DPDK 实现了对 IPv4 数据报文和 IPv6 数据报文的高性能的转发, 可以处理大规模的流量请求。IPv4/IPv6 地址翻译器参考 RFC6146 标准, 可以实现从 IPv6 网络到 IPv4 网络的流量翻译, 通过端口复用, 实现 N:1 的地址映射。

关键字: IPv6 过渡 有状态协议地址翻译器 DPDK 地址翻译技术

DESIGN AND IMPLEMENTATION OF IPv4 / IPv6 NETWORK ADDRESS TRANSLATOR BASED ON DPDK

ABSTRACT

With the exhaustion of IPv4 address and mature development of IPv6 protocol technology, IPv6 commercialization is imminent. However, the existing equipment cannot upgrade directly from IPv4 to IPv6 because the format of IPv4 address and IPv6 address are not compatible. IPv4 networks cannot access into IPv6 networks directly. In this situation, IPv4/IPv6 translation technology is much important. With the help of IPv6 transition technologies seamless, the IPv6 networks could replace IPv4 networks without the awareness of user. In the same time, the server bandwidth of concurrent connections is raised to the level of gigabit per second because the global Internet traffic is increasing continuously. The performance of the server is limited. It is not available to resolve the high concurrent flow with the promotion in hardware. People began to improve the performance of concurrent data flow from multi-ways, such as the implementation of servers, the organization of databases and even the working mode kernel.

By studying the stateful address translation technology and high-performance concurrent data flow technology, this paper implements a simple IPv4 and IPv6 packet forwarding plane and stateful IPv4/IPv6 address translator on Intel DPDK (Data Plane Development Kit) platform. IP forwarding platform achieves high performance forwarding of IPv4 and IPv6 packets by using DPDK platform. IPv4/IPv6 address translator with reference to RFC6146 can translate the traffic from IPv6 network to IPv4 network. Translator also achieves N:1 mapping address by reusing ports.

KEY WORDS: IPv6 transition, stateful protocol address translator, DPDK

目 录

第一章 绪论	1
1.1 课题背景	1
1.2 课题意义	3
1.3 论文的研究目标和创新点	5
1.4 论文的组织 and 结构	6
第二章 背景技术介绍	7
2.1 IPv6 介绍	7
2.1.1 IPv6 的报头字段	7
2.1.2 IPv6 扩展首部	8
2.3 网络层的地址解析协议介绍	9
2.3.1 ARP 协议	9
2.3.2 邻居发现协议	11
2.4 DATA PLANE DEVELOPMENT KIT (DPDK) 高性能平台简介	14
2.5 有状态地址翻译技术	17
2.5.1 NAT64 方案中的地址翻译规则	17
2.5.2 NAT64 技术的域名解析过程	18
2.5.3 NAT64 的应用场景	19
第三章 IPV4/IPV6 地址翻译器的需求分析	22
3.1 IPV4/IPV6 地址翻译器的设计动机	22
3.2 IPV4/IPV6 地址翻译器的需求	23
3.2.1 网络转发平台的设计要求	23
3.2.2 有状态协议地址翻译器的设计要求	24
第四章 IPV4/IPV6 地址翻译器的设计与实现	25
4.1 IP 转发平台的设计与实现	25
4.1.1 系统模块的设计	26
4.1.2 IPV4 转发平台的设计	29
4.1.3 IPV6 转发平台的设计	32
4.2 有状态的 IPV4/IPV6 地址翻译器的设计	39

4.2.1 IPv4/IPv6 地址翻译器与 IP 转发平台的结合 40

4.2.2 数据库模块..... 40

4.2.3 协议翻译块..... 42

第五章 IPV4/IPV6 地址翻译器的部署与测试 44

5.1 测试目的 44

5.2 环境搭建和测试流程 44

5.3 测试拓扑和测试项目 45

5.3.1 连通性测试..... 46

5.3.2 业务验证测试..... 46

第六章 总结与展望..... 48

6.1 全文总结 48

6.2 工作展望 49

参考文献..... 50

致 谢..... 52

硕士期间论文发表..... 53

第一章 绪论

1.1 课题背景

互联网的诞生与发展标志着全球进入了一个全新的信息时代，今天的人们已经习惯利用以互联网为载体进行日常的生产生活。随着近些年来移动互联网和云计算等新技术的发展，以 IPv4 为基础的互联网络已不能满足现有设备的接入需求。IPv4 作为现在互联网的基础协议，其缺点也逐渐暴露出来^[1]。

首先，IPv4 地址空间具有局限性和地址分配不均匀的特点。IPv4 地址长度为 32 位，每个 IPv4 地址包括 2 部分：网络地址和主机地址。网络地址标识了主机属于哪一个网络，主机地址唯一地定义了网络上的主机。这种层次结构在带给 IPv4 地址好处的同时也导致了 IPv4 地址危机的产生。尽管 IPv4 地址的总数接近 40 亿，但是由于 IPv4 地址分配的不均匀性和互联网爆炸式的增长，使得地址被快速消耗。在这种背景下，一些缓解 IPv4 地址危机的办法开始广泛使用，其中包括网络地址转换(NAT)技术和无类域间路由(CIDR)技术^[2]。一方面 NAT 技术和 CIDR 技术不能从根本上解决 IPv4 地址不足的问题，另一方面，IP 地址规划结构变差，路由表更多，互联网骨干线路路由器压力越来越大，从而限制了互联网规模的进一步发展。

其次，由于 IPv4 格式的限制，使得 IPv4 的性能已经不能满足现有的网络要求。IPv4 协议是 20 世纪 70 年代开发的，目的是为美国政府，研究机构 and 高校之间建立通信和信息共享机制。由于当时接入规模的限制，开发者并未考虑到协议的安全性、可用性和服务质量等问题。原有的 IPv4 协议已经不能满足现有的互联网络的需求，尽管人们在后期对 IPv4 协议做了一系列的修补和改进，但这并不意味着现在的 IPv4 协议是一种最佳的实现，也不意味着仅仅通过简单的修补就能使得 IPv4 协议满足现有的网络需求。

IPv6 的开发是基于 IPv4 的开发及应用所获得的丰富经验，其保留了经过实践所获得的大量有效机制，摒弃了现有 IP 协议的局限，并且通过引入新的机制极大的扩展了协议的可用性和灵活性^[3]。

由于在 IPv4 设计之初，并未考虑到网络的移动性，使得 IPv4 协议缺少对移动场景的支持。在移动网络中，所有发送到移动节点的数据报文都需要通过家乡代理路由，导致家乡网络负载过重，同时由于 IPv4 在地址上的匮乏性使得

每个移动 IPv4 都要有外地代理, 出现三角路由的现象, 使得整个网络的复杂性大大增加。

最后, 在 IPv4 协议设计初始, 仅仅支持静态配置, 用户需要对所有接入网络中的主机分配 IP 地址。随着 ISP 的出现, 主机可以开始通过 DHCP 协议对 IP 地址进行动态配置。但是 IPv4 依旧不支持地址的自动配置和发现。

IETF 在 1998 年提出 RFC2460^[4], 将 IPv6 作为下一代 IP 网络的标准协议。IPv6 协议具有更大的地址空间和更好的扩展性, 可以有效地解决 IPv4 地址所带来的问题。然而, IPv6 协议和 IPv4 协议并不兼容, 从 IPv4 全面透明的升级到 IPv6 需要大规模的投资和建设。面对如此大规模的非盈利的成本消耗, 网络运营商和内容服务提供商都对 IPv6 的发展持观望的状态。在未来很长一段时间内, 在保证网络正常访问和用户体验的情况下, 网络运营商需要以最小的成本和代价, 逐步地从目前基于 IPv4 协议的互联网络平滑过渡到下一代以 IPv6 为架构的网络。

同时, 随着 Web2.0 和移动互联网的兴起, 整个互联网已经从单一的用于科研的内部网络, 发展成了一个应用于各行各业承载海量信息的全球网络。全球各地的人们想要通过互联网随时随地获得信息, 互联网中的每一台服务器每时每刻都要处理数以万计用户请求。这就对内容服务提供商提出了新的要求: 高并发性和高可扩展性。随着用户规模的增加和网络流量的爆炸, 现有的网络服务器已经不能满足高流量的并发性的用户请求, 大规模的访问量对服务器的性能提出了非常高的要求。更为重要的是, 互联网受众的广泛性, 使得网络服务的访问量增长速度非常大, 网络服务系统必须具有良好的可扩展性。

但是目前的情况缺不容乐观, 现有的网络服务器在处理大规模用户并发请求的时候, 往往出现效率低下和设备瘫痪的状况, 导致用户得不到优质的服务质量。这被称为 C10K 问题^[5], 即服务器怎样快速处理数以万计的并发请求。在许多年的探索过程中, 人们逐渐发现, 解决 C10K 问题不能单单依靠设备硬件的升级。服务器硬件的升级对于服务器整体的并发性能的提高微乎其微, 服务提供商不能仅仅通过购买和升级设备解决这一问题。C10K 问题的根源在于后台处理数据的应用程序运行效率过低, 程序并行性较差。为了解决 C10K 的问题, 需要通过制定合适的数据包处理策略, 编写高性能的应用软件解决问题。最近几年, 在 C10K 的基础上, 人们又进一步提出了 C10M 问题^[6], 即服务器怎么样处理千万级别的并发请求。其不涉及服务器的硬件和软件, C10M 问题还从操作系统和网卡驱动的层面上对整个数据包处理流程进行了分析。

1.2 课题意义

随着 IPv6 协议标准的日趋完善和 IPv4 地址枯竭,使用 IPv6 协议取代原有的 IPv4 协议作为当今互联网的基础架构协议已经势在必行。由于目前绝大多数的信息源和用户都在 IPv4 上,从 IPv4 迁移过渡到 IPv6 需要一个相对漫长的过程。运营商需要更换通信设备以使整个网络同时支持 IPv4 和 IPv6,内容服务提供商需要开发新的 IPv6 应用,终端设备也需要在操作系统中支持 IPv6 协议,只有大家同时努力,才能加快整个 IPv6 网络的建设和普及。在 IPv4 过渡到 IPv6 的过程中,最大的问题就是两个 IP 协议版本不兼容。IPv4 和 IPv6 协议的不兼容性主要体现在如下方面:IPv4 和 IPv6 地址长度和结构的差异;IPv4 报文选项与 IPv6 报文选项的差异;IPv4 和 IPv6 寻址方式的差异;ICMP 协议的差异等都是 IPv4 协议与 IPv6 协议中不同的地方。这样的差异导致 IPv4 网络和 IPv6 网络不能直接进行通信。

如今,整个网络不仅仅要面对 IPv6 产业的布局和发展问题,还需要考虑如何循序渐进的,平稳地从以 IPv4 为基础架构的网络过渡到 IPv6 网络。同时,在保证过渡期间互联网络稳定和业务连续的同时,业界还需要考虑怎样尽可能地减少过渡产生的成本消耗。目前,IETF(Internet Engineering Task Force)等标准化组织已经针对上述问题提出了相关的技术解决方案,根据工作方式的不同,主要分为双栈技术,隧道技术和地址翻译技术三个方向^[7]。

双协议栈技术(Dual Stack)是指网络中的主机和节点同时运行 IPv4 和 IPv6 两套协议栈的一种过渡技术的实现方式。从技术层面上来说,双栈技术是其他翻译技术的基础。从目前的状态上看,双栈技术是目前最成熟的过渡技术。双栈技术仅仅需要设备同时支持 IPv4 和 IPv6 协议,而当前最新的主流操作系统均已经支持 IPv4 和 IPv6 双协议栈。但是目前互联网络中的大部分老旧设备只支持 IPv4 协议,要全部对这些设备进行升级,需要花费大量的人力物力。双栈技术包括 DS-Lite 技术和 6RD 技术。

隧道技术(Tunneling)是将一种协议封装到另外一种协议中的技术。隧道技术利用一种传输协议,将其他协议的数据报文封装在指定的报文中,然后在网络中进行传输,当数据包到达隧道的另一端时,隧道对数据包进行解封装并取出原有的报文。在 IPv6 过渡技术中,隧道技术将 IPv6 报文封装在 IPv4 报文,实现 IPv6 报文在 IPv4 网络中的透明传输。隧道技术一般由隧道出口,隧道入口,协议封装,乘客协议等几部分组成,如图 1-1 所示。隧道技术包括 6to4 技术和 ISATAP 技术等。

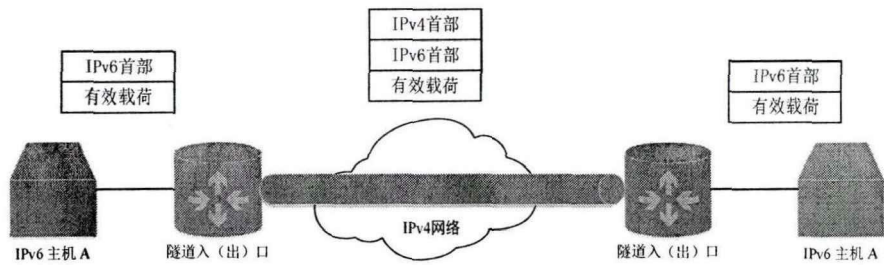


图 1-1 隧道的基本原理

翻译技术（Translation）是指通过算法使得 IPv4 报文与 IPv6 报文可以相互转换。利用部署在网关位置的地址翻译器，整个网络可以完成 IPv4 报文和 IPv6 报文之间的协议转换，经过协议翻译的报文可以顺利到达异构网络并访问异构网络中的设备。翻译技术主要基于翻译算法，通过对不同版本的 IP 报文的识别，重构对应的 IP 头部，使得 IPv4 网络中的主机和 IPv6 主机可以透明地通信。翻译技术分为有状态协议翻译和无状态协议翻译两种。前者在翻译过程中需要保存翻译过程中所产生的状态信息，而无状态翻译机制不需要翻译过程中的状态信息。

目前为止，面对各种需求不同的过渡场景，没有一种可以一劳永逸的通用的过渡方案能够覆盖所有的场景，解决所有的问题。不同的业务需求也需要选择不同的过渡方案来进行 IPv4 到 IPv6 的过渡。双协议栈技术作为其他所有过渡技术的基础，是最简单，兼容性最好的技术。双栈方案只需要网络中的所有设备同时支持 IPv4 和 IPv6 两种协议。但是，其部署成本大和设备升级消耗大，不能进行平滑过渡。隧道技术实现简单，部署灵活，但隧道技术没有实现异构网络间的互相访问问题，因此隧道技术的应用前景有限。翻译技术分为有状态协议翻译技术和无状态翻译技术，其能够真正的解决 IPv4 网络向 IPv6 网络的过渡问题，实现双向的异构网络之间的通信。无状态翻译技术运用无状态映射关系技术，但是对 IP 地址的格式有特殊的要求。通过将无状态翻译技术中的地址映射技术与有状态翻译技术相结合，可以充分发挥翻译技术的优势，支持复杂的网络应用场景。

对于网络并发性的问题，尤其是之前提到的 C10K 问题，工业界已经有不少完备的解决方案。通过使用高性能的数据报文处理框架，配合相关的硬件资源，服务器可以同时处理 10000 个来自客户端的并发访问请求。解决 C10K 问题的核心在于改变后台应用处理数据的方式。

经典的处理数据报文的方式一般使用同步阻塞的 I/O 处理模型，其开发周期短，维护简单，代码可读性强。传统的服务器处理程序通过建立多进程或多线程，处理每一个链接请求。对于每一个连接请求，服务器需要为其产生一个新的进程，每一个请求都在各自的进程内进行被处理。当需要进行数据库的查

找和写入时，服务器同样使用阻塞的 I/O 模型^[8]。对于上述模型，在少量请求连接数目的情况下，同步阻塞的 I/O 处理模型容易编写，使用方便。但是随着连接数的增多，其需要的进程或线程数目也随之增多，CPU 对于进程或线程的切换更加频繁。加上阻塞 I/O 的存在，使得服务器消耗大量的资源和时间用于切换过程，并发性降低，程序整体性能下降。

在此基础上，人们通过改进现有的软件架构，改善底层操作系统内核参数等方式，使得 C10K 问题得到解决^[9]。通过使用异步的非阻塞的 I/O 模型，建立线程池等方式，现代服务器程序的并发性能大大提高。现代服务器将每一个请求看作一个接一个的事件，每一个事件通过异步的方式，在不同的进程或线程中进行分发和处理。在处理完成后，进程池回收进程或线程。通过事件驱动服务器，业界改变了传统服务器处理数据包的方式，提高处理性能。

但是随着互联网，尤其是移动互联网的日益普及，应用服务器所需要处理的并发请求数目也随之大幅提高。因此，在 C10K 问题的基础上，人们进一步提出了 C10M 问题，即服务器怎样同时处理千万级别的并发请求。通过改进软件架构，改善系统内核等以往的做法已经不能同时处理千万级别的并发请求。人们需要一种新的数据包的处理方式，从操作系统和网卡驱动的层面上改变收发网络数据包的方式。针对上述问题，Intel 提出了 DPDK(Data Plane Development Kit)^[10]，DPDK 是 Intel 公司为旗下 Intel 网卡在 x86 和 x64 平台下开发的一款高性能的网络驱动组件。DPDK 提供了一系列基础组件，通过使用 DPDK，开发人员可以抛弃 Linux 内核，自己编写转发协议栈，在用户空间实现高性能的数据包转发。

1.3 论文的研究目标

本文基于地址翻译技术和 DPDK 技术实现一种 IPv4 网络过渡到 IPv6 网络的高性能解决方案。通过对有状态地址翻译技术和高性能并行技术的研究和实现，本课题实现了下列研究目标：

(1) 研究基于 DPDK 的高性能转发平台，探究怎么样改进现有的底层网络架构，从而实现数据包的高效转发。同时本课题结合之前的研究经验，在 DPDK 驱动上实现了基于 IPv4 和 IPv6 的转发功能。

(2) 研究与实现高性能的有状态地址翻译技，实现了一个 IPv4/IPv6 网络协议地址翻译器；

本课题分为 IPv4 和 IPv6 转发平台的实现和有状态的 IPv4/IPv6 网络协议地址翻译器的实现两方面内容。

1.4 论文的组织 and 结构

本论文的组织结构按章节分为下述的几个部分:

第一章主要介绍了本课题研究的意义和背景, 研究的目标和创新点等内容。

第二章主要介绍了地址翻译技术的中无状态地址翻译技术和 Intel 的 DPDK 高性能转发平台的相关背景和概念。

第三章介绍 IPv4/IPv6 地址翻译器的需求分析。

第四章详细介绍了转发平台的设计与实现和无状态协议地址翻译器的设计与实现。

第五章展望了本文提出的 IPv4/IPv6 地址翻译器的优势与不足, 对本课题未来进一步研究的方向进行了展望, 为今后相关学者进行后续研究打下了理论和实践基础。

每一个 IPv6 报文有且仅有一个基本首部，基本首部中含有整个数据报文载荷的基本信息。对于 IPv4 中的额外信息，IPv6 用扩展首部的方式进行实现。一个 IPv6 报文可以携带零个或多个 IPv6 扩展首部。通过 IPv6 扩展首部，IPv6 报文可以灵活地进行配置，提升报文经过中间节点的处理速度。最后是上层数据报文，这部分的性质和作用与 IPv4 一样，其是 IP 数据报文的有效载荷。

```

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Version| Prio. |                               Flow Label                               |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|                               Payload Length                               |
|                               Next Header                               |
|                               Hop Limit                                |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|
|
|
|                               Source Address                               |
|
|
|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|
|
|
|                               Destination Address                               |
|
|
|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

7

IPv6 报头是定长的。所以 IPv6 报头中删除了原有 IPv4 报头中的报头长度字段选项。IPv4 中巨包字段选项所实现的功能，在 IPv6 中均由扩展报头所程度，这种改变使得整个 IPv6 报文的组织结构更加清晰，处理效率更高。IPv6 报头中包含 128 位的源地址和目的地址，其彻底解决了 IPv4 地址空间不足的问题[11]。

在 IPv4 首部中存在数据报文分片处理相关的字段，其中包括标识符字段，标志字段和分片偏移字段。IPv6 协议则摒弃了这些跟分片相关的字段。分片处理是指发送方或中间节点将一个完整的数据报文分割成许多小尺寸的分片并将其重组成为数据包，以多个数据包的形式进行传输。当多个数据报文到达目的主机时，目的主机收到这些小的数据报文，将这些小尺寸的数据报文进行重组，以还原原有的大尺寸的数据报文。在 IPv4 数据报文的传输过程中，如果网络发现数据报文的大小大于该网络所能承载的最大数据传输单元（MTU），网络会自动对 IPv4 数据包进行分片处理。在整个分片过程中，如果其中某个数据报文丢失或损坏，则需要重传所有的数据报文分片，因此分片传输效率低下。在 IPv6 协议中，引入了 MTU 发现机制以避免数据报文被分片。主机利用 MTU 发现机制在发送数据包前了解到发送路径上最小的最大数据传输单元的数值，从而调整发送数据报文的大小，避免数据报文在传输过程中被分片。如果发送主机希望对 IPv6 数据报文进行分片，可以使用扩展报头。基于上述理由，IPv6 报头丢弃了原有的 IPv4 报文中的标识符，标志和分片偏移等与分片相关的字段。

同时，为了提高转发速度，IPv6 在报头中去除了校验和字段。在 IPv4 网络中，数据报文每经过一个中间节点，生存周期减 1，数据报文需要重新计算校验，这无疑增加了中间节点的负担，降低了数据报文的转发速度。如今，数据包在传输过程中出错的概率已经非常小，传输层协议含有校验和字段。因此 IPv4 中的校验和字段已经没有存在的意义。同时为了保证传输报文的低出错率，有效地检验报文的传输错误，IPv6 强制所有具有检验和字段的传输层报文均要在目的主机接收数据报文后做校验和。从本质上讲，IP 协议是一种尽力而为的传输协议，IP 报文负责传输，没有责任保持上层传输的无差错。

IPv6 报头使用流量类别(Traffic Class)代替了 IPv4 中的 TOS 字段，增强了特殊数据的处理能力，使得转发的中间节点更容易区分不同 IPv6 数据报文的优先级。

2.1.2 IPv6 扩展首部

IPv4 报头的长度不定。为了满足额外功能，IPv4 报头中含有各种额外选项，其中包括源路由选项，安全选项等。虽然 IPv6 协议舍弃了 IPv4 报头中的

很多字段，但是，IPv4 原有的功能都保留了下来。IPv6 协议中以一种全新的方式重构了 IPv4 报文。IPv6 协议使用了一种叫做扩展报头的机制。扩展报头位于 IPv6 基本首部和上层载荷数据之间，如图 2-2 所示。通过扩展首部的方式，中间节点能够以更快的速度处理数据报文。

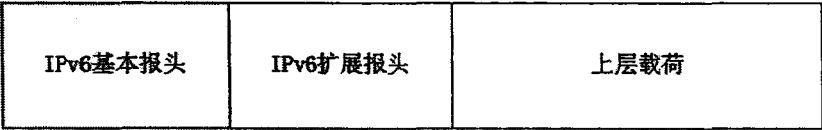


图 2-2 IPv6 扩展报头

IPv6 数据报文可以没有扩展报头，也可以有一个或多个扩展报头。如果一个 IPv6 有多个扩展报头，则所有的扩展报头必须按照规定的顺序在 IPv6 报文中出现。根据 RFC2460，一共有 6 种扩展报头，依次是逐跳选项扩展报头，路由扩展报头，分段扩展报头，目的选项扩展报头，认证扩展报头和加密安全净荷扩展报头。

- (1) 逐跳选项扩展报头携带可选信息。这些信息必须由传输路径上的所有中间节点进行检查并处理。逐跳选项扩展报头支持 IPv6 发送巨包。
- (2) 路由扩展报头中给出了若干传输路径上中间节点的信息。数据报文在到达目的地址前，必须访问这些中间节点。这在 IPv4 中被称为松散路由。
- (3) 分段扩展报头实现了 IPv4 的分片功能。
- (4) 目的选项扩展报头中含有仅能由目的节点所检查和处理的信息。
- (5) 认证扩展报头保证了传输数据的完整性，源主机和目的主机可以通过认证扩展报头来进行相互的认证。
- (6) 加密安全净荷扩展报头主要用于 IPv6 数据报文的加密和密钥的认证。

由于 IPv6 扩展报头按固定顺序进行排列，网络中间节点可以根据需要对数据包进行处理，从而减轻了中间节点的压力。同时，通过扩展首部，IPv6 协议的可扩展性和健壮性大大加强。

2.3 网络层的地址解析协议介绍

2.3.1 ARP 协议

在网络中传输数据报文的过程中，网络不仅仅要知道数据报文的网络层地址，还需要知道数据报文下一跳主机的二层链路物理地址，即 MAC 地址。在 IPv4 协议族中，ARP 协议为主机提供 IPv4 协议地址到物理地址之间的映射 [12]。ARP 的工作机制如图 2-3 所示

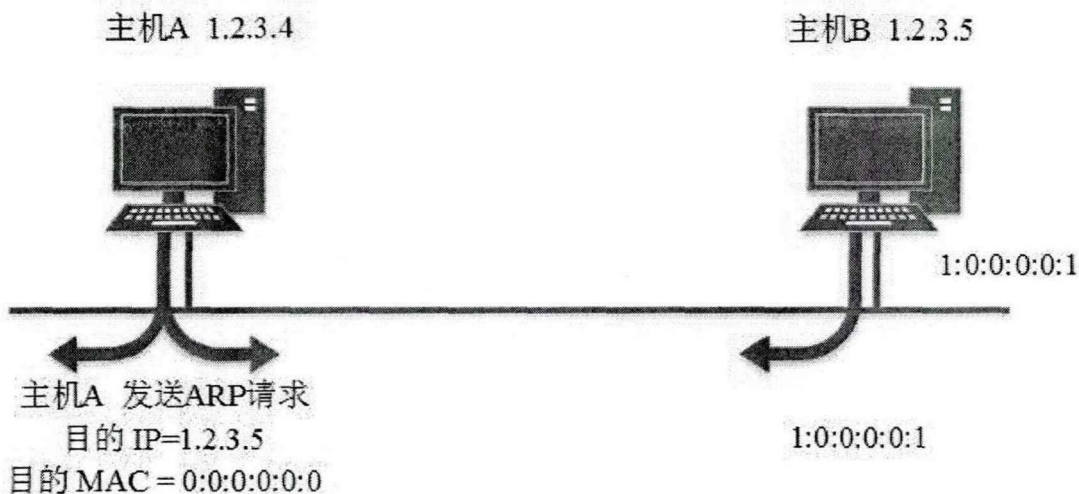


图 2-3 ARP 的工作机制

如图 2-3 所示，当主机 A 想要获得主机 B 的物理地址的时候需要经历如下步骤：

- (1) 源主机 A 获得目标的 IP 地址，主机 A 首先查找 ARP 缓存，如果 ARP 缓存中有目标主机 B 的 IP 地址，主机 A 直接构建以太网帧并发送。否则进入步骤 2。
- (2) 如果主机 A 不知道目标主机的 MAC 地址的情况下，主机 A 广播发送一个 ARP 请求报文。这个请求报文中包括了以下四部分：发送方的 IP 地址（1.2.3.4）和 MAC 地址，主机 A 想要知道的目的主机 IP 地址（1.2.3.5）和目的主机的 MAC 地址。其中目的 MAC 地址设置为全 0。
- (3) 接着 ARP 请求会在整个链路上进行广播。链路中的所有主机均能接收到 ARP 请求并进行解析。如果接收主机发现 ARP 请求报文中的目的 IP 与本机的 IP 地址符合，则进入步骤 3，否则丢弃数据报文。
- (4) 主机 B 接收到 ARP 请求报文，解析后发现 ARP 请求报文中的目的 IP 与本机 IP 一致，主机 B 首先会生产一个 ARP 应答报文，将应答报文中的发送方 IP 设置为本机的 IP 地址，发送方的物理地址填充为自己的 MAC 地址，目的 IP 和目的 MAC 设置为原来 ARP 请求报文中的发送方字段。最后主机 B 会更新自己的 ARP 缓存，并将 ARP 应答报文单播发送给主机 A。
- (5) 主机 A 收到 ARP 应答报文后获得相应的 MAC 地址。在更新缓存的同时，主机 A 会继续进行 IPv4 数据报文的转发。

因为 ARP 协议可以进行动态地址解析，所以在 TCP/IP 报文中，发送方只需要知道目标主机的 IP 地址，其物理地址可以动态获得。但是由于 ARP 请求报文是一种广播报文，如果每一次发送 IP 数据报文都要进行 ARP 动态解析，那么网络会出现严重的拥塞现象。因此，通常操作系统的做法是将获取到的 IPv4 地址和 MAC 地址相映射并缓存起来。当下一次再用到这个物理地址的时候直接从 ARP 缓存表中进行读取。ARP 报文的格式如图 2-4 所示。

0	7 8	15 16	BIT
HW INTERFACE TYPE		PROTOCOL	
HW ADDRESS SIZE	PROTOCOL ADDRESS SIZE	OPERATION	
SENDER HW ADDRESS			
		SENDER PROTOCOL ADDRESS	
RECEIVER HW ADDRESS			
RECEIVER PROTOCOL ADDRESS			

图 2-4 ARP 报文的格式

2.3.2 邻居发现协议

在 IPv4 时代,网络通过广播 ARP 报文来获取下一跳节点的 MAC 地址。但是 ARP 协议自身也有很大的缺陷。首先,ARP 协议中的 ARP 请求报文和 ARP 应答报文需要进行匹配。IPv4 主机接收到 APR 应答报文,不论主机以前是否发出过对应的请求,均要接收并更新缓存。这样就会导致虚假的 ARP 请求。其次,由于 ARP 请求数据报文是广播发送,导致局域网内的所有主机均要接收并查看广播的 ARP 请求报文,因此 APR 协议并不高效。最后,由于 ARP 在创造之初,并未考虑安全性的问题,因此 ARP 报文容易被劫持和仿造。整个 ARP 协议并不安全。因此,在 IPv6 协议中,为了完成物理地址的查询功能,IPv6 协议没有再重新设计一个链路层协议承担动态物理地址的查询任务,而是通过使用现有的 ICMPv6 协议,达到地址解析的目的。ICMPv6 的报文格式如图 2-5 所示^[13]。IPv6 使用邻居发现协议作为 IPv6 的物理地址查询协议。邻居发现协议定义在 RFC4861^[14]中。同时邻居发现协议进一步完善了局域网中主机的监测机制。使用 IPv4 协议的主机不能检测局域网其他主机是否可达,而在 IPv6 协议中不仅可以通过邻居发现协议检测周围邻居主机的可达性,还能通过此协议进行地址重复检测。

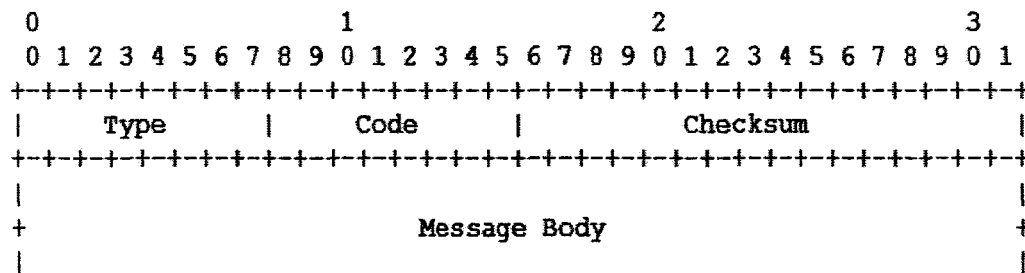


图 2-5 ICMPv6 报文格式

邻居发现协议一共定义了五种消息报文，其分别是用于路由器的 Router Solicitation 消息和 Router Advertisement 消息,用于普通主机的 Neighbor Solicitation 消息和 Neighbor Advertisement 消息,以及重定向的 Redirect 消息。

其中当 IPv6 主机发出 Router Solicitation 消息后，路由器会使用 Router Advertisements 消息对主机进行回应。Router Solicitation 消息主要用于路由器发现。Router Advertisements 消息用于无状态地址前缀的下发和 MIPv6 协议中家乡地址的配置。最后，MTU 发现机制也必须依靠 Router Advertisements 消息，主机可以通过此消息获得整个链路的 MTU 值以避免分片。Router Solicitation 消息和 Router Advertisements 消息格式如图 2-6 和 2-7 所示。

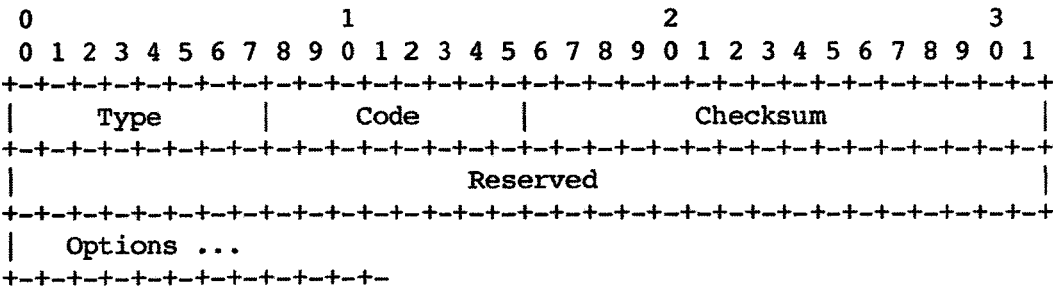


图 2-6 Router Solicitation 消息报文格式

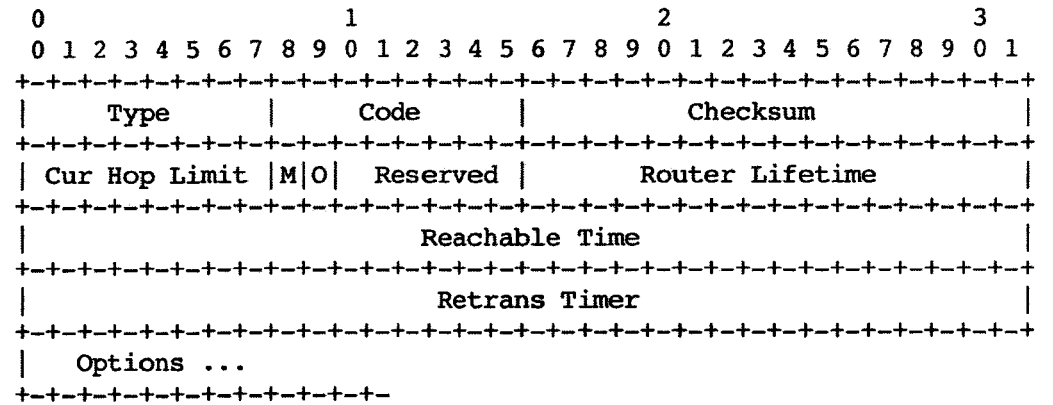


图 2-7 Router Advertisements 消息报文格式

Neighbor Solicitation 消息和 Neighbor Advertisement 消息用来代替原有的 ARP 协议，其消息格式如图 2-8 和 2-9 所示。IPv6 主机通过组播 Neighbor Solicitation 消息查询 MAC 地址。而主机通过接收 Neighbor Advertisement 消息得到相对应的二层物理地址。同时 Neighbor Solicitation 消息和 Neighbor Advertisement 消息还具有地址重复检测和邻居可达性的检测的功能。在 Neighbor Solicitation 消息中，如果目的地址是一个多播地址，源地址是单播地址，则邻居发现协议正在执行地址解析。如果目的地址是一个单播地址，则邻居发现协议正在进行邻居不可达检测。相比于无状态的 ARP 协议而言，邻居发现协议是一个有状态协议，其一共有五种状态，分别表示了地址解析和地址可达性的所有过程。五种状态如下所示。

- (1) INCOMPLETE 这个状态说明该地址正在执行地址解析。Neighbor Solicitation 消息已经向目标主机的请求节点多播地址发送请求报文，但是对应的 Neighbor Advertisement 消息还没有收到。
- (2) REACHABLE 这个状态说明在规定时间内受到了肯定的 Neighbor Advertisement 消息，邻居转发路径正常。
- (3) STALE 状态表示地址解析完成，并且已经更新了缓存中的链路层地址，地址正在被使用。并且在 STALE 状态下，可达性可以被迅速检验。
- (4) DELAY 状态是一种优化，它适用于在流量缺乏的场景下进行可达性的确认。
- (5) PROBE 状态是只在一段时间内通过发送 Neighbor Solicitations 消息来主动得搜索和确认邻居的可达性。

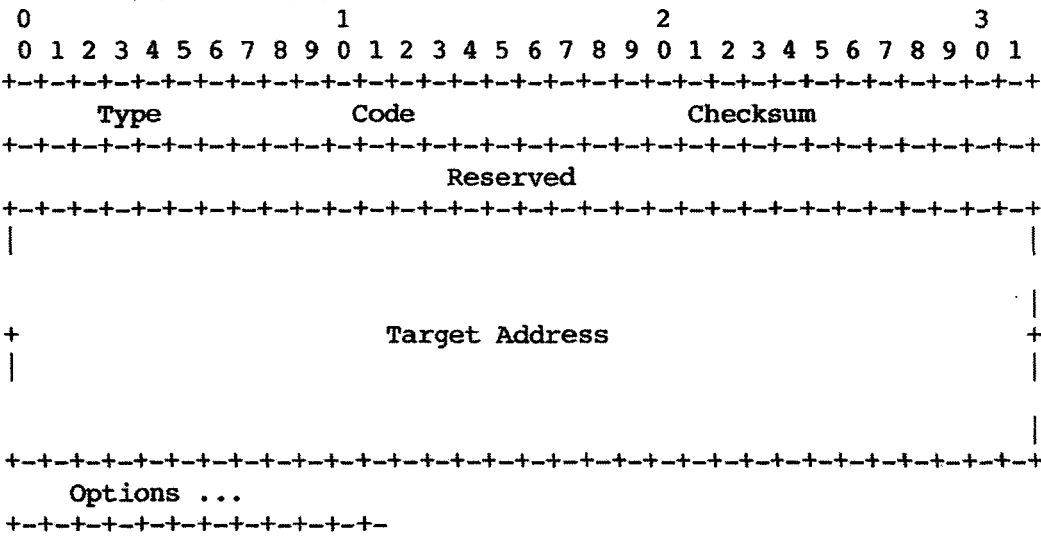


图 2-8 Neighbor Solicitation 消息报文格式

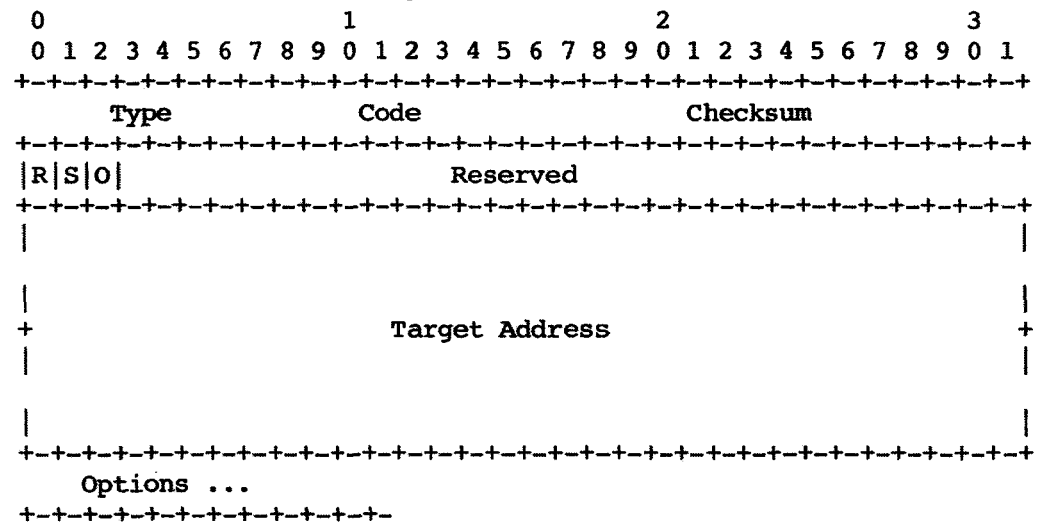


图 2-9 Neighbor Advertisement 消息报文格式

最后的 Redirect 消息用于优化传输路径。Redirect 消息通知节点一条到目的节点的更优路径的存在。同时，此消息还可以通知节点某个远端的目的主机实际上是局域网内的邻居节点。

2.4 Data Plane Development Kit(DPDK)高性能平台简介

随着 CPU 核心数量的增加，以及硬件的处理能力愈来愈强，如今网络处理平台的硬件计算能力大幅提升。业界已经达成共识，要充分利用多核能力，应用软件必须是多线程的，只有多线程的应用程序才能将硬件平台的能力发挥到最大^[15]。现代服务器大都是以 Unix 或者 Linux 为基础的计算平台。随着多核心处理器的出现，这些操作系统也利用多核处理器的优势对自身加以改进，适应现代处理器的需求。但是，最初 Unix 系统的设计初衷是成为一个电话网络控制系统，Linux 是一个类 Unix 操作系统，所以 Unix/Linux 从本质上并不是为高性能的并行计算而设计。因为控制系统主要针对的目标是多用户和长作业，其并没有针对大量并行数据做高性能的优化和处理。Unix/Linux 并不适合处理大规模的并行性数据服务。

Unix/Linux 内核处理数据的本质是通过中断实现的。在网络传输中，大量数据报文被网卡接收，通过 DMA 的方式，直接传送到操作系统预先规划好的供内核态使用的内存中。然后操作系统对报文进行中断处理，更新接收描述符，将报文送入网络堆栈，网络堆栈再对数据进行校验和与解包，根据数据报文的地址做出分析。如果是目的地址是本地套接字，或需要进行本地处理，则内核再将数据报文复制到用户空间。反之，数据报文从用户空间到最后被物理网卡发送的过程中，用户程序通过操作系统调用和软中断的方式将数据写入用户缓冲区，再由用户缓冲区复制到系统内核缓冲区，接着网络堆栈对数据报文进行封装和处理，通知并将新的数据从内核缓冲区复制到网卡的缓冲区中，最后网卡刷新换缓冲区并发送用户生成的报文^[16]。

在上述过程中，随着网卡接收数据报文的增多，整个系统的负荷将会增加，中断处理所消耗的时间会上升，其严重影响 CPU 的使用效率。同时，中断会引起上下文切换，增加内存的负担，高速缓存也会被用户空间大量的系统调用所污染。

此外，即使有 DMA 的存在，数据报文的输入输出也会是性能上的瓶颈。从之前的描述中我们可以发现，网络数据的处理需要多次的内存拷贝，其不仅仅需要从网卡内存到系统内存，还需要跨内核空间和用户空间的内存管理。而且由于一般网卡内存较小，在大量数据报文到达的情况下，更会增加中断次数，降低整个系统的性能。

DPDK 是 Intel 提供的 Linux 上的数据平面的快速数据报文处理框架，其提供了一个低开销的方法来替代传统 LinuxTCP/IP 协议栈，使得用户可以快速开发出高性能的用户态数据平面业务以替代传统的 Linux 系统调用^[17]。与原始 Linux 程序相比，DPDK 平台减少了中断和上下文切换的次数，精简数据复制路径。通过以上方式，使用 DPDK 编写的程序具有更好的性能。DPDK 之所以具有良好的性能，得益于以下几点：

首先 DPDK 在处理数据报文时，采用轮训机制，而非中断机制。在轮训机制下，整个程序能够高速的接收和发送数据报文，CPU 可以时刻关注数据报文的到达。其次 DPDK 将所有程序放置在用户态，使用数据平面进行收发数据报文，为此，DPDK 绕过了系统内核，当网卡接收数据报文之后，DPDK 直接将数据报文从网卡内存转移到用户缓冲区内。这意味着在数据传输的过程中，数据报文的处理不经过操作系统内核，直接由网卡到达用户空间，不与系统内核进行交互，从而精简数据处理流程和数据传输路径。

另一方面，DPDK 在初始时刻就会分配好对应的内存缓冲区，应用程序可以避免在运行过程中向系统请求内存空间。由于系统将内存分配给用户程序后并不关心用户如何使用这块内存，因此 DPDK 程序在网卡接收到数据报文后，直接将数据报文放置到应用程序的缓冲区中，原地处理。在发送的时候，DPDK 程序也直接从应用程序的缓冲区中获取数据报文。从而 DPDK 绕过了系统内核，实现一个零拷贝的过程。DPDK 的整体模型如图 2-11 所示。

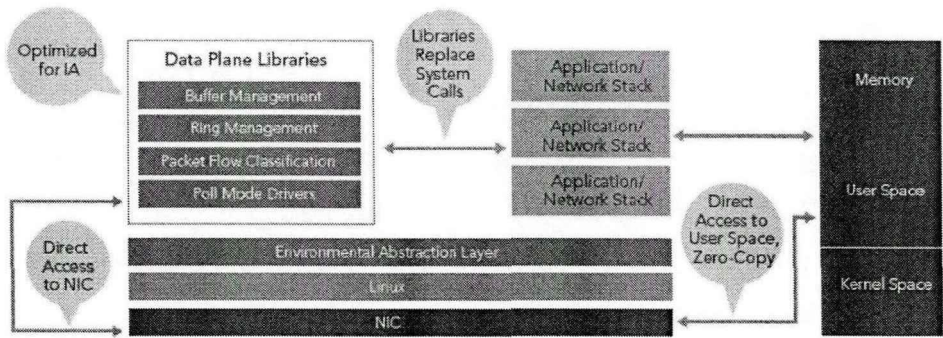


图 2-11 DPDK 模型

同时由于 DPDK 是 Intel 开发并提供支持的，其在多核心处理器上做了进一步优化。DPDK 应用程序在运行时刻可以规定具体使用哪一个核心处理器，并且每个核心处理器有自己的管理句柄，应用程序可以将网卡端口与 CPU 的各个核心分别绑定，利用 CPU 的亲合性，使得每一个核心只负责一个网卡端口的数据的发送和接收。通过这种方式，DPDK 打造了一个高效紧凑的用户态运行环境。DPDK 的主要结构如图 2-12 所示：

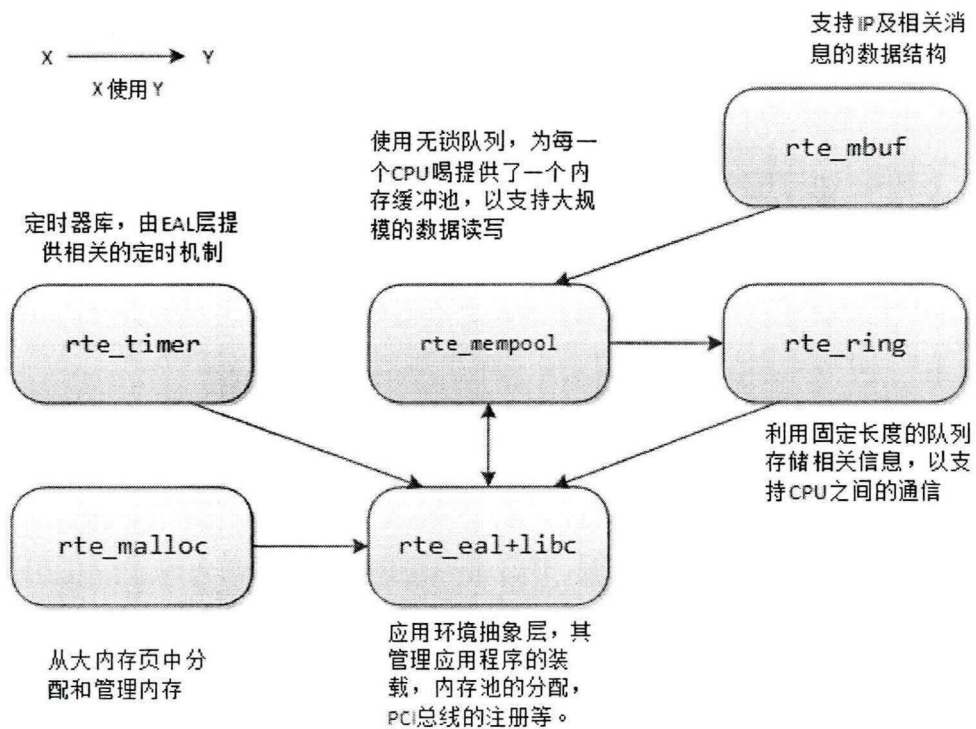


图 2-12 DPDK 主要结构

DPDK 主要由以下几部分组成：

EAL(the Environment Abstract Layer): EAL 提供了一个通用的接口以用来隐藏硬件实现。通过 EAL，用户程序有权限访问和分配系统的硬件资源。EAL 作为 DPDK 的核心组件还负责应用程序与内存等硬件之间的链接，是所有程序使用 DPDK 的根本组件。

内存管理(RTE_MEMPOOL): 内存管理组件提供了分配和管理内存的 API。内存管理主要用于程序的内存的分配和数据报文的产生。通过无锁队列，内存管理优化了每个核心 CPU 的本地缓存，内置了内存数据对齐的帮助指令^[18]。同时在内存管理组件中，使用了一种叫做大页(hugepage)的技术。大页技术是指一个内存页超过 4K 的页表。总所周知，一般的内存页大小为 4K，在这种背景下，随着系统内存的增大，映射表的条目将会增加。随着指令的不断执行，内存页的查找和复制将会影响 CPU 的执行效率。但是由于内存大小是固定，只能通过增加单页表的尺寸以加快页表的查找效率。大页技术由此而来。通过大页技术，程序在使用程序局部性原理的状态下，提高了 CPU 的效率。

队列管理(RTE_RING): 在 DPDK 平台中内存池和数据路径的队列均由 RING 模块进行管理。DPDK 在实现过程中借助 RING 模块的可以同时被多个核心 CPU 操作的特性，达到内存共享的目的。队列管理支持生产者消费者模型。为了加快处理速度，DPDK 中的队列是一个无锁的实现。

报文管理(时钟管理 RTE_MBUF): 报文管理模块提供了分配和释放报文的接口。

时钟管理(RTE_TIMEER): 时间模块可以获取相关的 CPU 频率, 以便异步执行回调函数。

2.5 有状态地址翻译技术

RFC 6146^[19]标准描述了一种有状态的IPv4/IPv6地址翻译技术。其叫做有状态 NAT64(Stateful NAT64)。RFC 6146标准定义了一种有状态的映射表, 使得 IPv6单栈主机可以主动向IPv4单栈主机发起访问请求。

有状态的 NAT64 的访问流量是单向的, 只支持 IPv6 主机主动向 IPv4 服务器发起的链接请求并从 IPv4 服务器主机获取请求应答。有状态 NAT64 需要保存 IPv4 地址和 IPv6 地址翻译前后的状态信息, 因此 NAT64 方案建立了一张绑定状态表。其可以记录在地址转换地址期间 IPv4 地址和 IPv6 地址的绑定关系。通过有状态的记录表项, NAT64 方案可以实现 N: 1 的地址端口映射。多个 IPv6 地址和端口可以映射到同一个 IPv4 地址的不同端口上。通过这种方式, 可以减少 IPv4 地址和 IPv6 地址的使用, 不造成地址的浪费, 在 IPv4 地址匮乏的情况下, 加大了 IPv4 地址的利用效率。

在普通情况下, 地址翻译技术只涉及网络层的地址翻译和转换, 并不会设计到上层载荷。但是由于一些上层协议涉及到网络层的数据, 因此在地址翻译转换的方案中还需要考虑对传输层及上层载荷中部分信息的翻译和填充问题, 以保证整个数据报文携带信息的一致性, 使得报文被最终目的主机正确接收。由于 IPv6 和 IPv4 地址的不同, 域名解析服务器需要不同的条目以对应不同的地址类型。在一般情况下, NAT64 方案不仅仅需要部署地址翻译器, 还需要有相对应的域名解析服务器。IPv4/IPv6 地址翻译器执行 IPv6 与 IPv4 之间的地址翻译, 而域名解析服务器用于实现 IPv4 网站域名所对应的 32 位的 IPv4 地址向 128 位的 IPv6 地址的解析过程。

2.5.1 NAT64 方案中的地址翻译规则

NAT64 中的 IPv6 地址是一个嵌入式的 IPv6 地址, 由于 IPv4 和 IPv6 地址长度的差距, 在翻译方案中的 IPv6 地址组成由前缀和 IPv4 地址构成。这个 IPv6 地址必须是可逆的, NAT64 可以从 IPv6 地址中恢复出原有的 IPv4 地址。NAT64 地址转换方案支持多种地址转换算法。同时 NAT64 翻译方案也必须支持由 RFC6052 标准定义的地址转换方案, 其格式如图 2-13 所示^[20]。

PL	0	--	32	40	48	56	64	72	80	88	96	104	---
32	前綴		V4(32)				u	后綴					
40	前綴			V4(24)			u	(8)	后綴				
48	前綴				V4(16)		u	(16)	后綴				
56	前綴					(8)	u	V4(24)			后綴		
64	前綴						u	V4(32)				后綴	
96	前綴										V4(32)		

图 2-13 RFC6052 中规定的 IPv6 地址格式

图 2-13 中的前缀字段表示 IPv6 地址中的前缀字段。IPv4 地址的嵌入位置随着 IPv6 地址前缀长度的不同而变化。在 RFC6052 中，前缀长度的值只能在 32，40，48，56，64，96 中进行选择。

图 2-13 中的标志位 u 在第 64 位开始偏移，占位 8 比特。其与 IPv6 地址结构中的 u-bit 兼容。

图 2-13 中的 V4 标志表示 32 位的 IPv4 地址，其一般用于标识目的主机。由于 IPv6 前缀长度不定，因此，IPv4 地址可能会被 u 标志位隔开。

IPv6 后缀在 IPv4 地址之后，一般为全 0，是一个预留的标志位。在一些特殊情况下，用户可以在这些标志位中添加传输层端口号，或者填充翻译过程中的必要的状态信息。

2.5.2 NAT64 技术的域名解析过程

在 NAT64 应用场景中，IPv6 主机要访问 IPv4 服务器的前提是 IPv6 用户必须知道 IPv4 服务器域名对应的 IPv6 地址。但是大部分 IPv4 服务器并没有配置 IPv6 地址，因此，IETF 在 RFC6147 种定义了 DNS64 的概念^[21]。在 DNS64 规范中，如果 IPv6 用户使用域名解析服务，目标服务器的 IPv4 地址由 DNS 服务器提供并转换为 IPv6 地址。这个特殊的 DNS 服务器我们称之为 DNS64 服务器。DNS64 服务器需要放置在一个专门的位置，DNS64 服务器既能让 IPv6 网络中的用户访问，其自身还必须能访问 IPv4 网络中的域名解析服务器。当 DNS64 服务器收到来自 IPv6 用户对 IPv4 服务器的域名解析请求时，DNS64 服务器首先会查询相关的缓存记录，如果缓存记录命中，则 DNS64 服务器直接返回这个 AAAA 记录给 IPv6 用户。如果这个 AAAA 记录不存在，DNS64 服务器将会访问 IPv4 网络中的域名解析服务器，发送一条 A 记录的 IPv4 地址的查询报文。在获得解析后的 IPv4 地址之后，IPv6 将存有 IPv4 地址的 A 记录翻译成

为过渡技术能识别的包含 IPv6 地址的 AAAA 记录。最后 DNS64 服务器将 AAAA 记录返回给 IPv6 用户。

2.5.3 NAT64 的应用场景

NAT64 翻译方案典型的应用场景如图 2-14 所示。

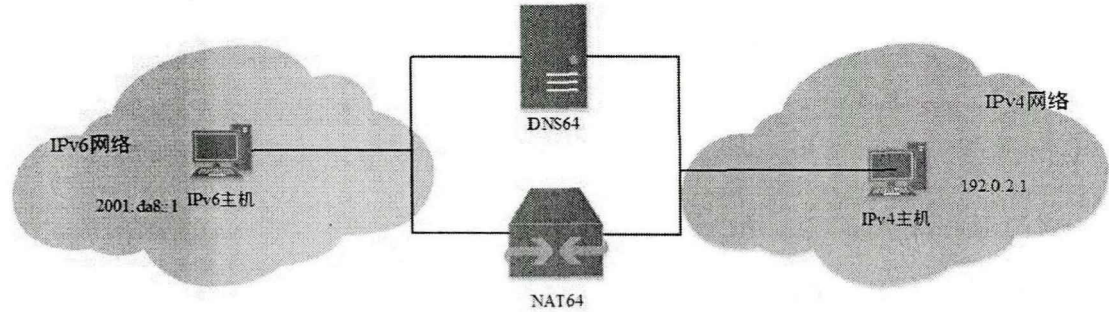


图 2-14 NAT64 翻译方案典型的应用场景

在 NAT64 的网络中，当 IPv6 主机想访问 IPv4 资源，IPv6 主机会发起访问请求并通过查询域名服务器获得 AAAA 资源记录。当 IPv6 主机获得一个能够被 NAT64 方案识别的经过转换的 IPv4 服务器的 IPv6 地址的时候，IPv6 主机向 IPv4 主机发起 TCP 链接，IPv6 数据报文路由到 NAT64 网关，NAT64 网关随机选择一个 IPv4 地址和一个没有用过的端口号作为转换后的 IPv4 源地址和 IPv4 源端口号，并对数据报文进行翻译，完成从 IPv6 网络向 IPv4 网络的报文转换。被转换的报文从 NAT64 网关的 IPv4 端口发送到目的 IPv4 主机。

接着 IPv4 主机收到访问请求，确认无误后会向 IPv6 主机发送 IPv4 应答报文，同样经过 NAT64 网关转换后返回 IPv6 应答报文。

在定义地址转换的方案同时，NAT64 标准还描述了一些额外的概念：

五元组：所谓的五元组是指源地址，源端口号，目的地址，目的端口号和相关上层协议标识。对于 TCP 和 UDP 来说，上层协议标识表明了所对应的协议类型。对于 ICMP 协议来说，上层协议标识表示了 ICMP 的查询编号。由于 TCP 是有状态的协议，因此，在每一次 IPv4 与 IPv6 的转换过程中，NAT64 方案均需要对 IPv4 地址和 IPv6 地址分别记录。

绑定信息表(Binding Information Base)：绑定信息表记录了相关的地址转换的映射关系。由于五元组的存在和端口的可重用性，NAT64 方案中所支持的每一种协议均对应有一张绑定信息表。现有的 NAT64 方案只支持 TCP，UDP 和 ICMP 协议。对于 TCP 和 UDP 所对应的绑定信息表来说，每一个绑定信息表项都记录了一个 IPv6 转换地址和端口到 IPv4 转换地址和端口的映射。对于 ICMP 而言，绑定信息表的表目分别是 IPv6 地址，ICMPv6 标识符和 IPv4 地址，ICMP 标识符。在 NAT64 运行的过程中，绑定信息表目总是动态变化的。

Session 表：一个 NAT64 地址翻译器还有多个 Session 表，每一个 Session 表对应 NAT64 所支持的一种可转换的上层协议。Session 表中的每一条表目都关联着一条相对应的链接的所有的状态。对于一条 TCP 或 UDP 链接，每到达一个新的 IPv6 报文，NAT64 就根据这个 IPv6 数据报文挑选一个新的 IPv4 地址和端口，从而将一个 IPv6 五元组映射到一个 IPv4 五元组上。对于 ICMP 协议，Session 表则是将 IPv6 源地址，目的地址，IMCPv6 标识符和 IPv4 的源地址，目的地址，IMCP 标识符关联起来。同时，每一个 Sesseion 表项都含有一个生存时间，整个 Session 表根据生存时间对表目做动态的删减。具体的 NAT64 方案中的数据报文处理流程如图 2-15 所示：

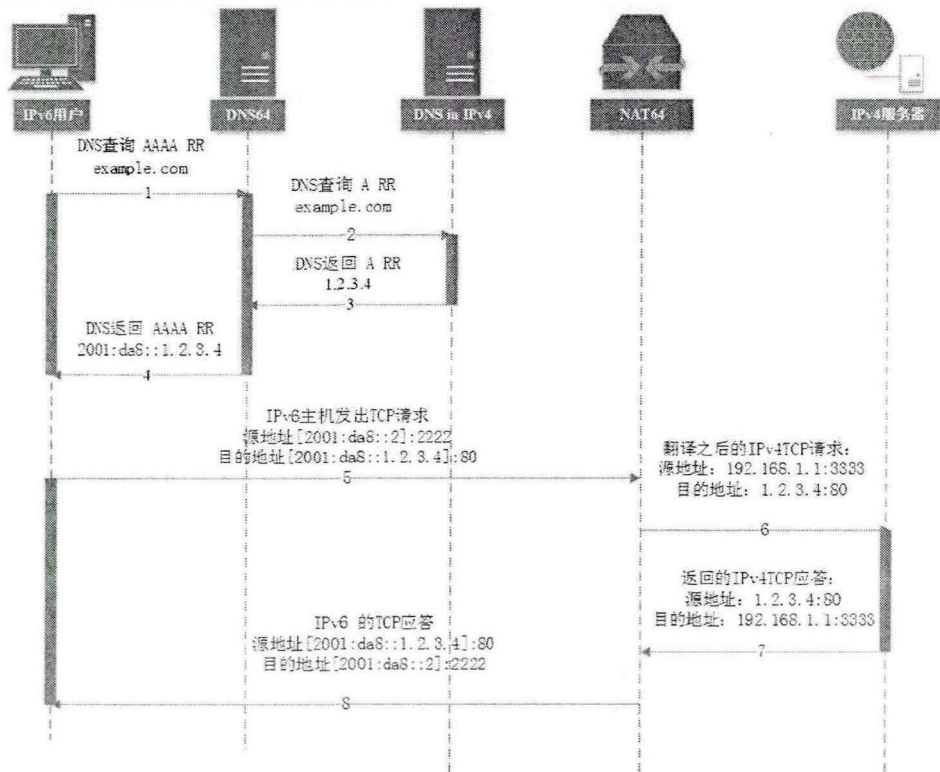


图 2-15 数据报处理流程

- (1) IPv6 主机向域名解析服务器查询 example.com 所对应的 IPv6 地址。
- (2) DNS64 服务器向 IPv4 网络中的域名解析服务器查询 example.com 的 A 资源记录。
- (3) IPv4 网络中的域名解析服务器返回一个 A 资源记录，其 IPv4 地址为 1.2.3.4。
- (4) DNS64 服务器将获得的 A 资源记录进行转换，生成可以被转换和识别的嵌入了 IPv4 地址的 IPv6 地址，即 2001:da8::1.2.3.4。
- (5) IPv6 主机获得地址之后向 IPv4 网络发送一个目的地址为 2001:da8::1.2.3.4，目的端口为 80 的 TCP 请求。这个 TCP 请求被默认路由到 NAT64 网关。根据 NAT64 的规则标准，NAT64 网关对目的 IPv6 地址进行转换，NAT64 转

换网关随机选择一个 IPv4 地址和端口号(192.168.1.1:3333)作为转换后的源地址和端口。并在信息绑定表和 Session 表中各添加一条信息。

- (6) 当服务器收到 IPv4 请求报文并确认无误后其返回一个 IPv4 应答报文。
- (7) IPv4 应答报文经过 NAT64 转换网关, 网关根据之前的绑定信息将数据报重新转换为 IPv6 协议的包头。
- (8) IPv6 主机接收到转换后的 IPv6 报文, 完成一次 NAT64 转换。

根据上面的数据报处理流程我们可以发现, DNS64 地址解析服务器在整个数据报文的传输过程中主要扮演一个提供有效 IPv6 地址的角色。其可以根据得到的信息将一个 IPv4 地址嵌入到一个 IPv6 地址中去。而 NAT64 网关主要负责地址转换和状态记录。NAT64 网关随机挑选一个 IPv4 地址和端口号, 并根据一定的规则将 IPv6 报文翻译成为 IPv4 报文。在翻译地址的同时 NAT64 还记录了翻译过程中的相关状态, 将相关信息记录到信息绑定表和 Session 表中。

对于 NAT64 网关, 其位置是在 IPv6 网络和 IPv4 网络的边界处。NAT64 网关必须有 2 个以上的网络端口, 其中一个端口必须运行 IPv6 协议, 此端口是作为 NAT64 网关的 IPv6 出口, 而另外一个端口必须运行 IPv4 协议, 此端口是作 NAT64 网关的 IPv4 出口。

第三章 IPv4/IPv6 地址翻译器的需求分析

3.1 IPv4/IPv6 地址翻译器的设计动机

校园网络与园区网络和普通的居民网络有显著的不同。园区网网络各部分功能较为明确,划分清晰,流量变化平稳。普通用户的网络流量虽然较大,但是网络接入较为分散,没有优先级的区别,网络出口升级容易。而对于校园网络而言,其不仅仅需要给内部学生提供一个良好的可供学习和生活的网络环境,还需要保证学校日常的科研和办公。学校科研任务较重,需要赋予科研,办公和教学等区域网络较高的优先级,而宿舍和学生使用的网络优先级较低。由于历史原因,大部分校园网内的 IPv4 出口带宽并不高,因此在高峰时期,学生宿舍的网络常常会出现严重拥堵的现象。随着 Web2.0 的兴起,Flash 等技术的流行,各大网站都使用了更漂亮和繁杂的界面吸引用户,这就不可避免的增加了单个页面的访问流量,并且视频流量在整个网络访问流量中所占比重越来越大,导致整个网络的流量呈直线上升。在不提高网络带宽的情况下,用户的体验实际上是呈下降趋势的。对于这种趋势,网络运营商是通过优化路由,增加小区接入带宽和整个网络的出口带宽来解决网络拥塞的问题,而这些升级的费用最终还是需要用户去承担的。但是对于学校而言,其并不是类似于网络运营商,以赚钱为目的的商业组织,学校没有足够的资金去升级整个网络。并且学校的人口密度远大于普通的居民住宅小区,这对学校的出口带宽就提出了更高的要求。

现阶段,国内的 IPv6 部署主要集中于运营商内部的实验网络和各大高校的校园网。对于运营商内部的 IPv6 网络,运营商主要用来做终端和各种网络设备的检验和测试。运营商内部的 IPv6 网络几乎没有真实的用户接入,没有实际的流量产生,网络本身不需要考虑性能上的优化和计费上的考虑。而对于校园网络,各大高校的 IPv6 网络中有丰富的内部资源,IPv6 的访问流量也远大于运营商内部的实验 IPv6 网络。但是由于主要的网络内容提供商没有提供 IPv6 网络资源。因此在校园网内部,IPv4 的流量仍旧远大于 IPv6 的流量,IPv6 网络的出口和入口没有 IPv4 网络那么拥塞。这就造成了一种资源浪费的现象,用户想要访问 IPv4 网络,但是 IPv4 网络出口拥塞,带宽资源较小,用户体验差,而

IPv6 网络恰恰相反, IPv6 网络建设完善, 出口带宽流畅, 流量较小, 使用率低。

因此我们需要一种技术, 在不更换和影响现有校园网络的情况下, 以最小的花费和代价使得校园网内部的 IPv4 网络和 IPv6 网络流量达到平衡。即, 通过 IPv6 过渡技术使得 IPv6 用户可以顺利访问 IPv4 的网络资源。通过这种方式, 使得 IPv4 网络的部分流量转移到 IPv6 网络中去, 在降低 IPv4 网络出口流量压力的同时, 提高 IPv6 网络出口的利用率。考虑到学校本身经费并不充裕, 因此需要提高设备的利用效率。而校园内部人口密度较大, 在高峰时段网络出口处的突发流量会突然增大, 需要提升网络的并发性能。

Intel 的 DPDK 是专门为高性能的网络转发和处理而设计产生的数据平面开发工具。Intel 在 x86 平台上对相关的网卡和 CPU 做了深层次的优化, 提供了一系列的开发工具。NAT64 技术已经于 2011 年被 IEFT 作为完善的 IPv4/IPv6 过渡技术并将其加入 RFC 标准。NAT64 标准方案提供了地址转换算法, 必要的数据结构, 方案同时对整体架构的参数配置和安全性做了详细的阐述。本课题结合上述原因和相关考虑, 在 x86 平台上, 通过 DPDK 开发工具, 遵循 NAT64 标准, 实现了一个 IPv4/IPv6 地址翻译器。用户可以通过地址翻译器实现 IPv6 主机访问搭建在纯 IPv4 之上的网络服务器。通过这个地址翻译器可以有效的转移部分 IPv4 流量, 使得整个 IPv4 出口出网络流量下降, 每个用户的平局带宽上升, 同时提高 IPv6 网络的利用率。

3.2 IPv4/IPv6 地址翻译器的需求

结合 3.1 节提出的 IPv4/IPv6 地址翻译器设计动机, 软件应有如下的要求。

3.2.1 数据转发平台的设计要求

由于 DPDK 高性能开发平台并没有相关的 TCP/IP 网络协议栈。因此 IPv4/IPv6 地址翻译器必须能够识别 IPv4 和 IPv6 报文, 使得其能够正常的接收和转发。

由于 DPDK 只能识别相关的网卡并获得 MAC 地址。因此, 地址翻译器必须能够为整个转发平台设计和设置相关的网络参数, 其中包括 IPv4 地址, IPv4 网络掩码, IPv4 静态路由, IPv6 地址, IPv6 链路本地地址和 IPv6 静态路由。地址翻译器是在 DPDK 平台之上进行构建的, 因此地址翻译器需要对操作系统的网卡, 内存和 CPU 核心进行设置, 包括所使用的网卡的阈值, 内存的可容纳的数据报的个数, CPU 的核心数目, 内存通道等。通过对预先规定相关的参数, 可以达到更好的适配性和通用性。

其次, IPv4/IPv6 地址翻译器还需要实现 IPv4 和 IPv6 网络的转发功能和路由查找功能。地址翻译器要能正确识别 IPv4 报头和 IPv6 报头, 对报头内的不同字段进行处理, 可以根据相关的规定和条件创建新的 IP 报文。由于地址翻译器所在的网络环境物理拓扑变化不大, 所以地址翻译器只需要实现静态路由条目的查找和存储, 可以根据到达的数据报文的 IP 地址和掩码实现最长前缀的匹配。

除了上述功能外, 对于 IPv4 而言, 还需要实现 ARP 功能。因为 IP 数据报文的转发需要在每一跳更改链路层报头的物理地址, 对于 IPv4 报文, 操作系统通过 ARP 协议获得局域网内下一跳的 IPv4 主机的物理地址。因此, 要想实现 IP 数据报文的转发, 必须要实现 ARP 协议, 通过 ARP 协议知道下一跳目的主机的物理地址。对于 ARP 协议, IPv4/IPv6 地址翻译器要实现以下功能: 翻译器能够根据 IPv4 报头的相关信息产生能被其他操作系统正确解析的 ARP 请求报文, 能够接收和识别其他操作系统发送的 ARP 请求报文, 能够对 ARP 请求做出正确的应答。最后, 地址翻译器需要为 ARP 协议配备一个 ARP 缓存, 这个 ARP 缓存能实现基本的增删改查功能。

对于 IPv6 网络, IPv4/IPv6 地址翻译器要实现邻居发现协议中的 Neighbor Solicitation 消息和 Neighbor Advertisement 消息。翻译器要能够正确识别和处理 IPv6 扩展报文。因为邻居发现协议底层依托的是 ICMPv6 协议, 地址翻译器要能够识别处理 ICMPv6 协议, 依据 ICMPv6 报文类型的不同分别作对应的处理。同时对于邻居发现协议, 因为 NAT64 网关本质上并不是一个路由器, 因此不用实现 Router Solicitation 消息和 Router Advertisements 消息。由于工作量的关系, 本课题也没有计划实现 Redirect 消息。与 IPv4 中的 ARP 缓存一样, 转发平台还需要实现一个缓存数据库实现 IPv6 地址和 MAC 地址之间的有状态的映射。

3.2.2 有状态协议地址翻译器的设计要求

IPv4/IPv6 地址翻译器要实现 NAT64 标准中的相关规定, 结合之前的 IP 转发平台, 实现一个有状态的 NAT64 地址翻译网关。通过 DPDK 开发工具提供的接口, 接管 Linux 系统中的网卡, 编写地址翻译程序。当数据流量报文进入系统后, 由底层的转发平台进行路由和判断, 当数据报文符合相关的配置, 地址翻译器完成 IPv6 地址到 IPv4 地址的转换, 并记录相关的状态信息。对于地址翻译器, 其要求与对应的 RFC 标准相符合, 实现相关的表目和状态记录, 从而完成有状态的 TCP, UDP 和 ICMP 的转换。

第四章 IPv4/IPv6 地址翻译器的设计与实现

在之前的论述中，论文对 IPv4/IPv6 地址翻译器中各个组件的相关概念，需要实现的协议组件和每个组件的实现要求行了分析和阐述。在上述论述的基础上，本章将阐述在 Linux 环境中对 IPv4/IPv6 地址翻译器的组件设计和原型实现。详尽解析对应的软件结构，报文的处理流程和每个模块之间的关系。整个系统分为 IP 转发平台和地址平台，下面分别对其加以介绍。

4.1 数据转发平台的设计与实现

由于本课题是在 DPDK 平台上实现 IPv4/IPv6 地址翻译器，而 DPDK 没有 IPv4 和 IPv6 协议，因此，地址翻译器首先需要实现一个 IPv4/IPv6 数据平面的转发平台。地址翻译器通过转发平台可以实现完整的 IP 数据报文的解包，修改与转发。同时，地址翻译器还需要实现路由表目的匹配和查找以便数据报文可以正常的在网络中流动。DPDK 平台的数据路径并不经过系统内核，DPDK 平台接管网卡后，数据报文从网卡直接转移到用户空间，其转移过程与操作系统本身没有任何关系。DPDK 平台自己搭建了一套网络数据报文处理流程，绕开了操作系统。因此我们还需要配置系统的相关参数，通过相关的注册函数完成对网卡，内存和处理器的配置。由于整个 DPDK 平台是采用轮询的方式对网卡进行监听，在处理数据报文时采用的是多次处理一次发送的原则，每个数据报文的接收和发送路径都与一个核心的 CPU 相互绑定。

整个数据转发平台的软件架构如图 4-1 所示，其主要包含了底层 DPDK 平台，系统模块，IPv4 转发模块和 IPv6 转发模块。其中每个模块内部还有若干小模块。其中系统模块中包括了参数解析模块，内存管理模块，端口配置模块，地址管理模块和二层报文处理模块。在 IPv4 转发模块中含有 IPv4 数据报文解析模块，IPv4 静态路由模块，ARP 模块与 IPv4 校验和模块。在 IPv6 转发模块中含有 IPv6 数据报文解析模块，扩展头处理模块，IPv6 静态路由模块，ICMPv6 数据报文解析模块，邻居发现协议模块与 IPv6 校验和模块。

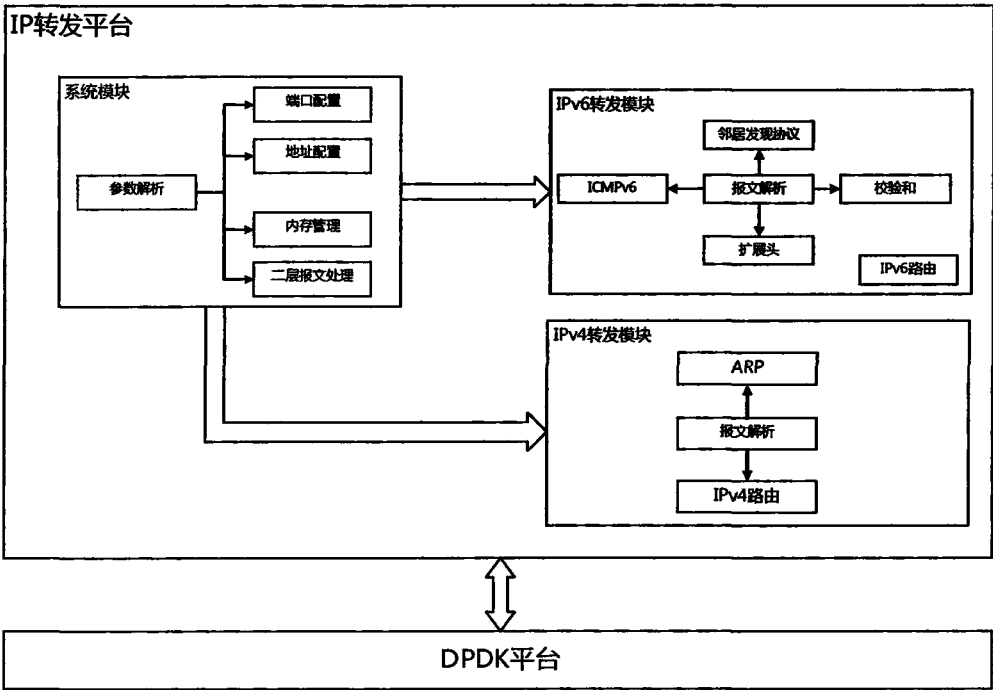


图 4-1 数据转发平台的软件架构

4.1.1 系统模块的设计

系统模块主要负责转发平台与 DPDK 平台的链接。通过系统模块，IP 转发平台可以对 DPDK 平台进行配置和优化。IP 转发平台中的每一条数据路径都与一个 CPU 核心相关联。系统模块定义一个全局变量 lcore_conf 结构，整个转发平台使用这个全局变量结构将所有的数据流量，缓存，配置与 CPU 核心关联起来。所有模块需要的信息均从此结构中获得。这个变量结构在整个程序中起了链接桥梁的作用。lcore_conf 结构如下所示。

```
struct lcore_conf {
    uint16_t lcore_id;
    uint16_t n_rx_queue;
    uint16_t tx_queue_id[MAX_PORTS];
    struct lcore_rx_queue rx_queue_list[MAX_RX_QUEUE_PER_LCORE];
    struct mbuf_table tx_mbufs[MAX_PORTS];
    struct Route_table *route;
    struct lookup_table *lookup;
    struct mempool *pool;
    struct Address *address;
}__rte_cache_aligned;
```

可以从上述结构体中我们可以轻易地发现，结构体不仅仅关联 CPU 的核心 ID(lcore_id)，接收队列描述符(rx_queue_list)，发送队列描述符(tx_queue_id)，

还关联了对应的数据报文内存(tx_mbufs)，路由表(route)，所有的缓存数据(lookup)，内存管理(pool)和地址管理(address)。这样统一的一个变量使整个程序的耦合性适中。每个模块之间既有相互联系，有彼此独立。如果以后要对程序进行扩展，我们只需要在 lcore_conf 结构中添加或删除相关模块的参数即可。

系统模块中的参数解析模块主要负责解析外界传入的相关参数，对 DPDK 平台和硬件进行配置。由于 DPDK 开发工具给予了编程人员极高的自由度，因此在带来性能提升的同时，也增加了整个平台的配置和参数解析难度。

参数模块通过标准 POSIX 的 API 实现参数的解析。程序的命令行选项如下所示：

```
-p PORTMASK -P --config(port,queue,lcore)[,(port,queue,lcore)]
  [--enable-jumbo] [--no-numa]
```

-p *PORTMASK* 选项可以设置端口的掩码从而进行端口的选择。

-P 选项配置所有端口是否为混杂模式和多播模式。在混杂模式下，网卡会忽略报文的物理地址，接收网络上的所有数据报文。在多播模式下，网卡是会接收具有多播物理地址的数据报文。IPv6 的邻居发现协议就是使用多播地址来进行地址解析的。

--config(port,queue,lcore)[,(port,queue,lcore)]选项将端口，数据路径和 CPU 核心绑定在一起，使得数据报文可以固定地由一个或是多个 CPU 核心进行处理。这样的好处在于系统可以减少 CPU 之间的通信，使得全部的系统资源全部放在数据报处理上，提高资源的使用效率。

[--enable-jumbo] [--no-numa] 是 2 个可选项，其分别对应的是程序是否可以处理巨包和程序是否是运行在 NUMA 架构上的。巨包是允许网卡接收长度超过 1518 字节的数据报文。使得单个数据报文所能承载的上层数据报文容量变大，数据报文的个数下降，从而降低设备因小包而造成的不必要的开销。NUMA 架构是一种分布式的存储器的访问方式，其可以大幅度的提高并行性。在 CPU 的核心数多余 8 的时候，使用 NUMA 架构的机器的性能会明显提升。

由于 DPDK 平台将绕过了系统内核，DPDK 平台把系统所有的内存都暴露给了程序员。因此在系统模块中，我们添加了内存管理模块，通过内存管理模块，我们可以动态的分配和调整所有的缓存和数据库大小，对内存进行精细化管理。程序以 CPU 插口为单位为每一个 CPU 核心分配对应的内存。同属于一个 CPU 插口的 CPU 核心共享一片内存区域。同时，我们把控制数据报文所使用的内存与流量数据报文的报文所使用的内存分开，使不同类型的数据使用不同的内存空间。由于 IP 报文转发平台是为 NAT64 网关服务的，因此，IP

转发平台不会自动产生流量报文。但是对于地址解析, ICMP 错误消息等控制报文, 却需要程序自己产生。为了提高处理流量报文的速度并且使得整个程序的结构更加清晰, 我们使用了两个内存池, 一个用来存储和发送流量数据报文, 另一个用来创建和发送控制数据报文。我们把数据流量路径放置在第一个内存池, 流入转发平台的所有数据流量都被网卡放置在此内存池中, 数据报文的解析原地进行。当对数据报文修改完成时, 数据报文原地进入发送队列等待发送。

众所周知, 频繁的内存的分配和回收会极大的影响整体系统的性能。因此课题根据现有程序采用了一种提前分配内存的方式, 使得内存的大小在一开始就固定下来。课题之所以这样做原因有如下几点: 首先, 在 IP 转发平台中, 当程序运行一段时间达到平稳后, 在网络拓扑变化不大的前提下, 经过转发平台的流量是在一定范围内浮动, 从而我们可以预估出经过网关数据平面的数据的流量上限。从而为每个 CPU 核心分配一个适当的内存大小。同时对于 ARP 缓存和邻居发现协议缓存, 我们也进行提前的固定分配。由于我们需要对 ARP 缓存和邻居发现协议缓存中的数据进行高性能的查找, 配合固定大小的数据库, 我们使用了 DPDK 平台自带的哈希表的 API 接口, 通过计算 ARP 缓存和邻居发现协议中的相关字段我们可以得出对应的哈希值, 并将其添加入哈希表目中。完成地址的查找工作。哈希表的初始化由 DPDK 给出的相关 API 完成, 每个哈希表表目的添加对应一个 `rte_hash` 结构。IP 转发平台通过 `rte_hash_parameters` 结构对哈希表进行配置。在 `rte_hash_parameters` 结构中分别定义了哈希表的名字, 哈希表使用桶的个数, 哈希键值的长度, 哈希函数和 NUMA 架构下对应的 CPU 接口。在 DPDK 中, 每一个哈希表与一个 CPU 插口相互对应。其在保证 CPU 共享内存的同时也减少了多核心状态下内存访问冲突的频率。

由于 IP 转发平台使用的是静态路由。因此, 整个路由表是在程序运行期间是不变的。整个路由表目由三部分组成, IPv4 或 IPv6 地址, 掩码, 地址对应的转发端口。路由表的查找使用 LPM 算法(最长前缀匹配算法)。与哈希表目一样, 一个 CPU 插口对应一个路由表。

最后是系统模块中的端口配置模块。通过外界的参数, 程序可以对整个服务器或者网关的所有端口进行选择性的使用和配置。在选择使用端口的同时, 程序通过 `rte_eth_dev_configure` 函数对端口进行配置, 将发送队列, 接收队列和 CPU 核心 ID 绑定在一起。转发平台还可以获得每一个端口的物理地址, 以便为之后的地址解析做准备。

最后要介绍的是二层链路的转发模块。为了提高全体数据的处理转发效率, 我们使用了一种叫做 `run-to-completion` 的报文处理模型。此模型将接收队

列的描述符，发送队列的描述符和 CPU 核心绑定在一起。数据报文从接收到发送不会经过别的 CPU 核心，使得整个处理器的所有核心的工作量是一样的。并且由于使用 RTC 模型，整个工作流量是不会被中断的。

对于数据的发送策略，我们使用了多次接收一次发送的概念。即不断地对网卡进行轮询，检查是否有新的报文到达，如果有新的报文，立即接收并处理。但并不要求处理完成的报文立即发送，而是当发送队列中的报文的数量达到一个上限值时，发送队列将这些报文一起发送出去。因为对于大量报文而言，如果是处理完成立即发送报文，不停的发送报文会直接影 CPU 的处理效率。将报文一起发送可以在最大程度上降低处理器在每个数据报文上花费的周期，减少不必要的内存访问。同时通过使用数组的方式来存储相关的数据报文，使得内存中所有的数据边界是几乎对齐的，这样整个转发平台就可以通过软件的方式对硬件进行优化。并且在大量数据报文同时到达的情况下，报文处理完成等待的时延可以忽略不计，对报文的转发没有任何的影响。同时为了防止在低流量的情况下，数据报文发送延迟过大，我们设置了一个最大延迟时间，当最大延迟时间到来的时候，无论发送队列中有多少报文，系统均将这些报文全部发送。

4.1.2 IPv4 转发平台的设计

对于流入 IP 转发平台的 IP 数据报文，IP 转发平台做如下处理。首先，IP 转发平台通过链路层报文判断和确定数据报文类型。如果是 IPv4 报文，其将会对报文做如下处理，示意图如图 4-2 所示。

根据 RFC1812 对 IPv4 数据报文的有效性做检查，包括对数据报文的长度的检查和 IPv4 报文字段值的校验的检查。

通过解包取得 IP 数据报文的的目的 IP 地址，进行大端格式向小端格式的转换。IPv4 转发平台使用最长前缀匹配的方式进行静态路由，获得端口和下一跳的 IPv4 地址。

在 ARP 缓存中查找下一跳的 IP 地址对应的物理地址。如果查找不成功，则发送 ARP 请求报文，对 IP 地址进行解析。如果成功则直接执行下一步。

得到下一跳 IP 地址对应的物理地址，在验证物理地址格式正确后。转发平台对数据报文链路层的地址进行更改，将原有的目的物理地址替换为解析出的新的物理地址，将源物理地址替换为发送端口的物理地址。

对数据报的 TTL 选项和校验和选项进行更改。由于经过网关，所以转发平台把 TTL 值减 1，同时为了保证报文的校验和正确，IPv4 数据报文的校验和加 1。

根据发送报文的端口号码，更改数据报文二层链路的源物理地址。
将相关报文放入发送队列，等待发送。

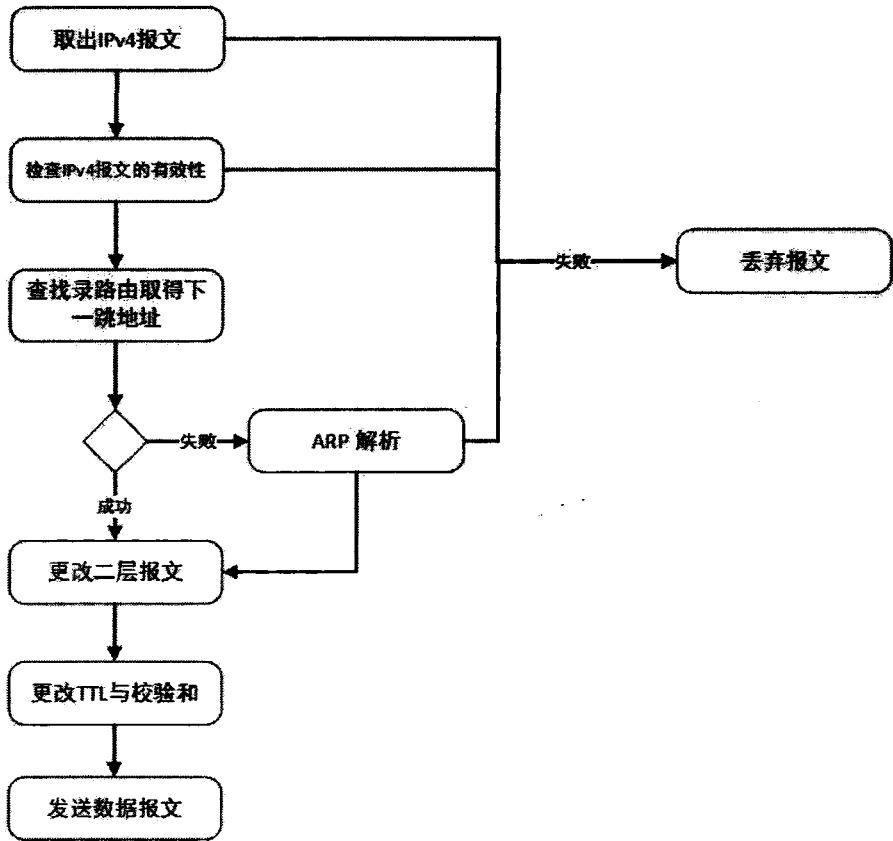


图 4-2 IP 数据报文处理流程

在 IPv4 转发模块中，主要功能还包括 IPv4 静态路由的查找与使用。转发平台使用 DPDK 所提供的 `rte_lpm_lookup` 函数，根据 IPv4 的目的地址和端口号对下一跳地址和端口进行查找。如果找到对应的端口号码和 IPv4 地址，则进行 ARP 缓存的查找，否则使用源端口号进行 ARP 缓存的查找。

IPv4 转发模块中最重要的是 ARP 协议的解析与实现。系统可以通过 ARP 协议完成对二层物理地址的解析。主机通过发送 ARP 请求报文来获取目的物理地址。通过 ARP 应答报文来响应其他主机发送过来的 ARP 请求报文。ARP 协议是 IPv4 数据报文转发过程中必不可少的协议之一。对于 ARP 报文，协议格式的定义如下所示：

```
struct ether_arp
{
    uint16_t    ar_hrd;        /* format of hardware address*/
    uint16_t    ar_pro;        /* format of protocol address */
    uint8_t     ar_hln;        /* length of hardware address */
    uint8_t     ar_pln;        /* length of protocol address */
    uint16_t    op;            /* ARP opcode (command)      */
}
```

```
struct ether_addr sha; /* the mac address for sender */
uint32_t spa; /* the ip address for the sender */
struct ether_addr tha; /* the mac address for the target */
uint32_t tpa; /* the ip address for the target */
};
```

这个结构体中的所有字段与 RFC 文档中标准的 ARP 格式一一对应。对于 ARP 报文，处理流程如图 4-3 所示。

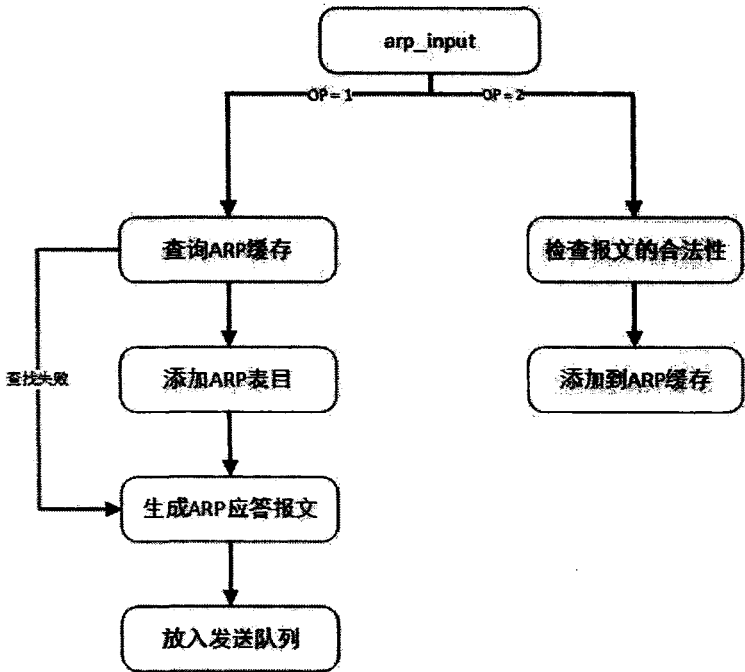


图 4-3 ARP 报文处理流程

对于到达的 ARP 数据报文，ARP 处理程序首先会解析 ARP 报文中所有字段，通过检查 ARP 的 OP 字段，判断 ARP 报文类型。如果 OP=1，说明是 ARP 请求字段。则进行如下操作。

首先会对整个数据报文中的所有字段进行提取，检查发送者的物理地址，IP 地址，发送者请求的物理地址和 IP 地址。由于 ARP 请求报文是一种广播报文，因此本机有可能会收到自己发出的 ARP 请求报文。如果发送者的物理地址和 IP 地址是本地地址，主机直接丢弃 ARP 请求报文。如果 ARP 请求报文中请求的物理地址不为全 0，主机也要丢弃该报文。

在解析完 ARP 请求报文后，ARP 模块会查询缓存，检查是否之前进行过相关字段的存储。如果之前没有进行过存储，ARP 模块将发送者的 IP 地址和物理地址存入 ARP 缓存以便以后的使用。然后 ARP 模块对 ARP 请求报文按规范进行修改，将自己的物理地址填入报文，生成对应的 ARP 应答报文。最后 ARP 模块将 ARP 报文放入发送队列，完成 ARP 请求报文的解析。

如果 OP=2，说明此报文时 ARP 应答报文，对于 ARP 应答报文，首先程序检查数据报文中的各个字段，尤其是目的物理地址字段和目的 IP 字段。模块需要确定应答报文是不是自己发出的。如果是，则继续处理，否则丢弃报文。之后 ARP 模块直接进行 ARP 缓存的更新，由于 ARP 应答报文和 ARP 请求报文并不配套，所以在处理 ARP 应答报文的时候，还需要同时对比缓存中的数据，以减少 ARP 报文欺骗的发生。

4.1.3 IPv6 转发平台的设计

相比于 IPv4 数据报文转发平台，IPv6 报文扩展头和邻居发现协议更为复杂，IPv6 数据报文转发平台的处理项目比 IPv4 数据报文转发平台更多。IPv6 数据报文转发平台的处理流程如图 4-4 所示。

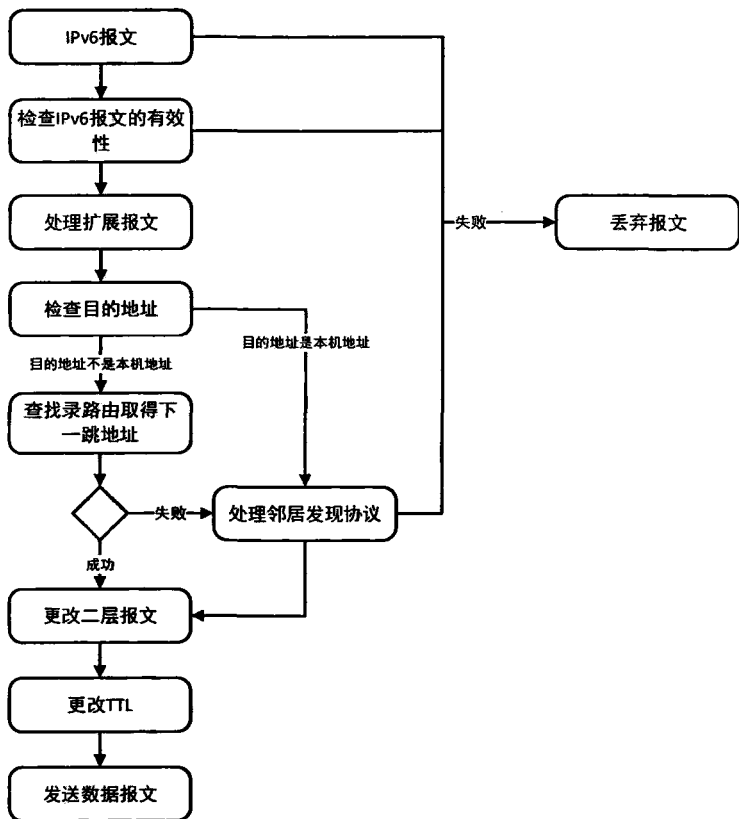


图 4-4 IPv6 数据报文处理流程

对比图 4-2 中 IPv4 的数据转发流程，我们可以清晰的发现，IPv6 数据报文的处理流程与 IPv4 数据报文的处理流程很相似。其区别主要在于对扩展报头的处理和将 ARP 协议处理模块更改成为了邻居发现协议处理模块。因此，对于整体的 IPv6 数据报文处理和路由查找过程，本文不做过多的介绍。下面主要介绍 IPv6 扩展报文的处理，ICMPv6 数据报文的解析和邻居发现中地址解析和地址重复检测的功能。

IPv6 扩展报文在 IPv6 报头中出现的次序是一定的，每种类型都有一个对应的代码值，每种报文中都含有长度信息。因此，在处理报文的时候。我们使用了循环的方式处理除了 IPv6 基础报头之外的所有上层载荷协议。每一次循环都会检查报文中 Next Header 中的协议代码值。当且仅当 Next Header 上层载荷协议是 TCP，UDP 或 ICMPv6 中的一种，循环处理结束停止为止。IPv6 上层载荷协议对应的代码值如表 4-1 所示。

表 4-1 IPv6 上层载荷协议对应标识代码

协议类型	代码值
TCP	6
UDP	17
ICMPv6	58
逐跳选项报头	0
目的选项报头	60
路由报头	43
分段报头	44
认证报头	51
封装安全净荷报头	52

除了分段扩展报头以外，大部分的 IPv6 扩展报头和上层的运输层协议的报文长度均是不固定的。在处理这些报文的时候，需要获得相关的长度值。对于 IPv6 扩展报文而言，其扩展报文的长度是以 8 字节为单位的，并且不包含第一个 8 字节。因此，在处理不定长度的扩展报头的时候，需要对报文中的长度字段进行转换，才能使用。

除了逐条选项报头以外，其他的所有扩展报头并不需要每一个中间节点检查和处理，即中间节点路由器只需要检查逐条选项报头和特定的目的选项扩展报头即可，其没有必要处理数据报文的其他上层信息。所以 IPv6 转发平台并没有对所有的扩展报头都做详细的处理，程序仅对逐跳选项报头与目的选项扩展报头做了对应处理。在逐跳选项中有选项类型字段，字段指定了当中间节点无法识别数据报文的时候该怎么样处理数据报文。字段的前 2 个比特规定了数据报文该怎么被处理。

- (1) 00: 忽略报文，继续处理。
- (2) 01: 丢弃数据报文
- (3) 10: 丢弃数据报文，并向发送此报文的源地址发送一个 ICMPv6 消息，代码类型为 2。

- (4) 11: 如果报文中的目的地址不是多, 丢弃数据报文, 并向发送此报文的源地址发送一个 ICMPv6 消息, 代码类型为 2。

下面论文介绍邻居发现协议的相关实现。由于是转发平台是作为上层 NAT64 地址翻译器的一部分所实现的, 因此转发平台不含任何的动态路由功能和重定向功能, 所以我们仅仅使用了邻居发现协议中的 Neighbor Solicitation 消息和 Neighbor Advertisement 消息。由于邻居发现协议并不像 ARP 协议一般是独立出来的, 其下层的承载协议是 ICMPv6 协议。由 ICMPv6 协议作为邻居发现协议的承载协议, 保证邻居发现报文的完整性和可识别性。ICMPv6 的数据结构定义如下所示。

```
struct icmp6_hdr{
    uint8_t icmp6_type;
    uint8_t icmp6_code;
    uint16_t icmp6_checksum;
    union{
        uint32_t data32;
        uint16_t data16[2];
        uint8_t data8[4];
    }icmp6_reserved;
}__attribute__((__packed__)); /* no more optimize */
```

其中 icmp6_type 表示 ICMPv6 的报文类型, icmp6_code 表示 ICMPv6 协议的代码值, 代码值的作用是可以进一步细化 ICMPv6 协议。icmp6_checksum 表示了 ICMPv6 报文的校验和, 在 IPv6 规范中, 上层载荷协议的校验和是强制执行的。icmp6_reserved 是指 ICMPv6 报文的报文正文。在 Neighbor Solicitation 消息和 Neighbor Advertisement 消息中, icmp6_reserved 字段要全部设置为 0。

论文首先介绍 Neighbor Solicitation 消息。Neighbor Solicitation 消息是地址解析请求消息, 如果 IPv6 主机已经知道对方的 IPv6 地址, 现在想通过 IPv6 地址进行物理地址解析, 获得对方的物理地址, 则主机需要发送一个多播的 Neighbor Solicitation 消息。Neighbor Solicitation 消息的结构如下所示:

```
struct nd_neigh_solicit{ /*neighbor solicitation*/
    struct icmp6_hdr nd_ns_hdr;
    struct ip6_addr tarip;
    /* may add options */
};
```

从上面的结构体定义中我们可以清楚的看到 Neighbor Solicitation 消息中包含一个 ICMPv6 报文和一个目标地址,其中目标地址是发送主机想要解析的 IPv6 地址。Neighbor Solicitation 消息的处理流程如图 4-5 所示。

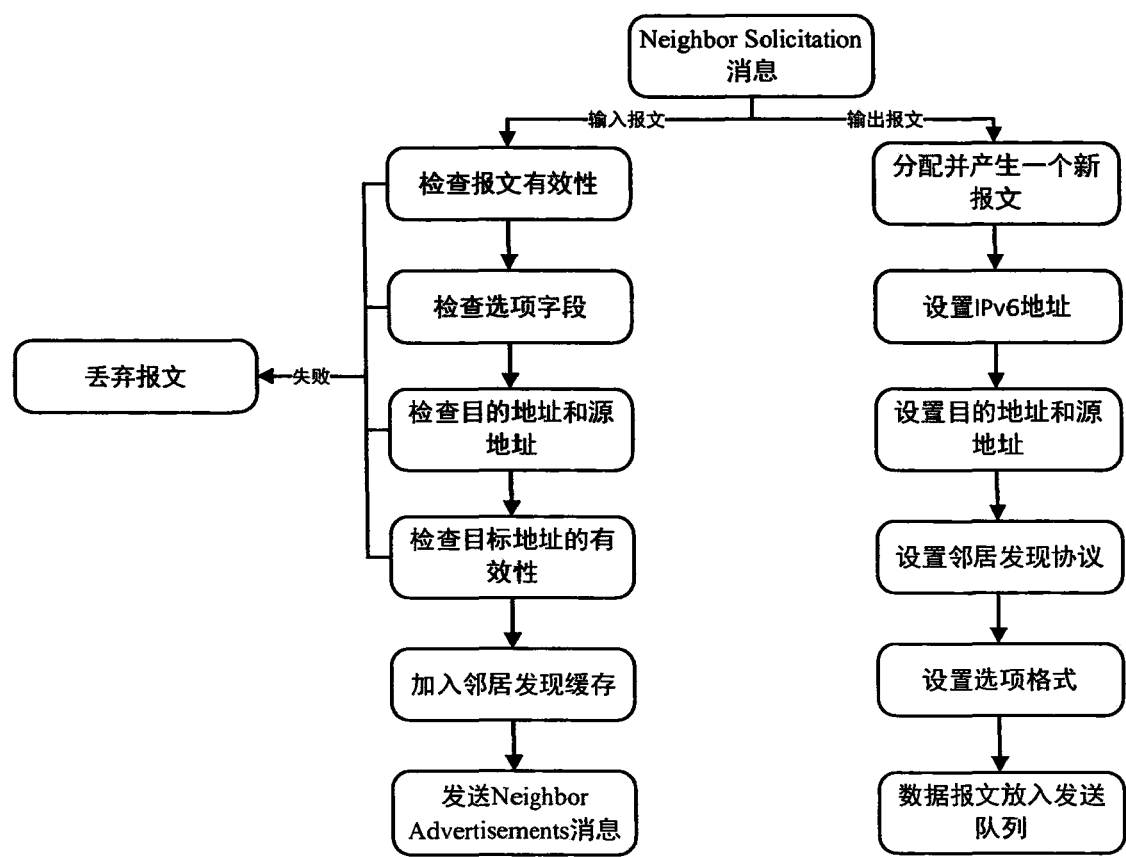
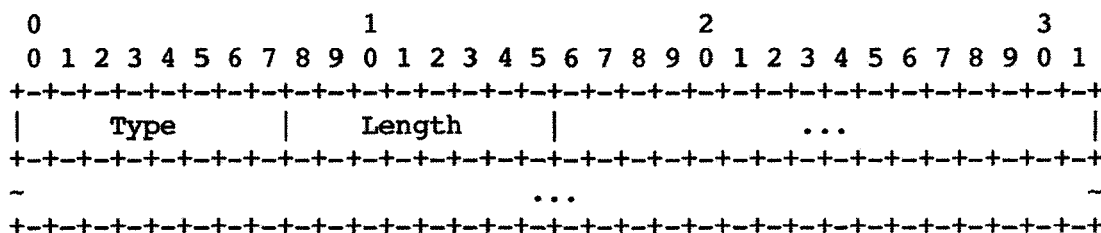


图 4-5 Neighbor Solicitation 消息处理流程

当转发平台收到一个 Neighbor Solicitation 消息时，IPv6 报文处理模块做如下处理：

首先，转发平台检查 IPv6 报文的完整性和各个字段值。解析 IPv6 数据报文。检查 IPv6 报头的有效性检查各个字段的有效性。同时报文要检查 IPv6 报头中的 Hop Limit 字段。因为 Neighbor Solicitation 消息在用于地址解析和地址重复检测的过程中，只在局域网内部传输，报文在各个主机之间只有一跳，中间不允许经过任何的中间节点。所以 ICMP 报头中的跳数字段的值必须是 255。

检查 IPv6 报头的源地址和目的地址。如果源地址为 IPv6 多播地址，数据报文是无效的。如果源地址为未指定地址，说明发送端正在进行 IPv6 地址的配置操作，发送主机正在执行地址重复检测。此时主机丢弃报文，不对该报文做任何回复。对于地址解析报文，模块要检查报文的地址，报文的地址必须是请求节点的多播地址。如果不满足上述条件，数据报文应该被丢弃。



源链路选项的数据结构定义如下:

当 **Type** 为 1 的时候表示源链路选项，里面包含了发送者的链路地址。但是如果报文的源地址是不确定地址，则不能含有源链路选项。

如果 Neighbor Solicitation 消息中有源链路选项，转发平台会检查邻居发现协议缓存。如果邻居发现协议缓存中没有对应的字段条目，则添加条目，如果有相关条目，则按照规则进行更新。

当主机收到一个 Neighbor Solicitation 消息，一般需要返回一个 Neighbor Advertisement 消息。这个 Neighbor Advertisement 消息格式需要按照 Neighbor Solicitation 消息中的字段进行构造。如果 Neighbor Solicitation 消息包含了链路层选项，且报文的目的地址是单播地址或任意播地址，说明发送端的邻居发现缓存中已经有目的主机的链路层地址，则在 Neighbor Advertisement 消息中可以省略目标链路选项。但是如果请求目的地址是多播地址，则 Neighbor Advertisement 消息中必须含有目标链路选项，同时说明发送端在进行地址解析。

转发平台发送一个 Neighbor Solicitation 消息时，IPv6 报文处理模块做如下处理：

(1) 首先为 Neighbor Solicitation 消息分配消息体并设置二层链路报文和 IPv6 报文。

- (2) 如果 IPv6 模块在执行地址解析或地址重复检查，模块会将 IPv6 的目的地址设置为请求节点的多播地址，源地址为本机地址。如果主机在执行地址重复检测，源地址必须是不确定地址，不添加源链路层选项。
- (3) 设置 ICMPv6 报文和 Neighbor Solicitation 消息。其中 ICMPv6 中的 Type 需要设置为 135，表示这是一个 Neighbor Solicitation 消息。
- (4) 根据需要添加源地址选项。包括类型，长度和发送者的链路地址。
- (5) 将报文请求的物理地址对应的 IPv6 地址加入邻居发现缓存，然后计算全部报文的校验和，并将数据报文添加至发送队列。完成 Neighbor Solicitation 消息的发送。

下面介绍 Neighbor Advertisement 消息。Neighbor Advertisement 消息的定义与 Neighbor Solicitation 消息类似。Neighbor Advertisement 处理整体架构如图 4-7 所示。

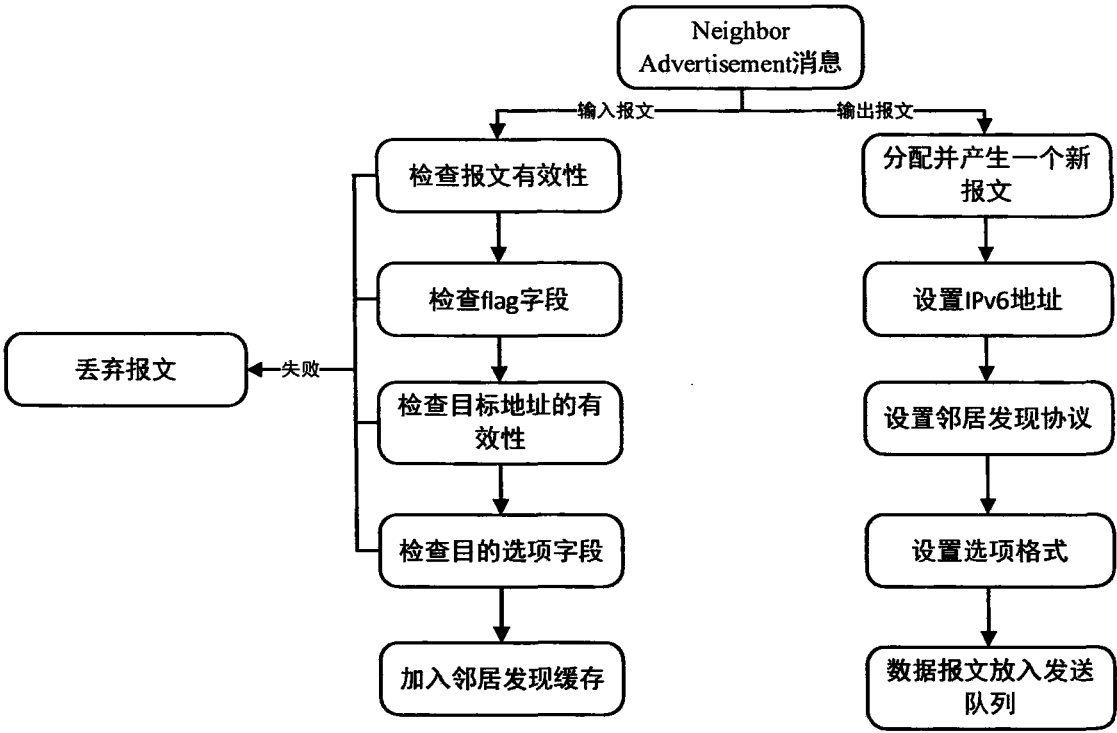


图 4-7 Neighbor Advertisement 消息处理流程

当转发平台收到一个 Neighbor Advertisement 消息时，IPv6 报文处理模块做如下处理：

- (1) 首先，转发平台解析 IPv6 数据报文。检查 IPv6 报头的有效性检查各个字段的有效性。检查 IPv6 报头中的 Hop Limit 字段。Hop Limit 必须为 255。其理由与 Neighbor Solicitation 报文中的原因一致。检查 ICMPv6 校验和是否为 0。检查 ICMPv6 的长度字段是否正确。

- (2) 检查 Neighbor Advertisement 消息中的 Flag 字段。从而判定如何对邻居发现缓存进行更新。在 Neighbor Advertisement 消息中的 Flag 字段一共有 3 个值, 分别是路由标志, 请求标志和覆盖标志。路由标志表示了发送者是路由器。请求标志表示该报文被发送以响应 Neighbor Solicitation 消息。覆盖请求则指出了邻居发现协议是否要强制更新缓存。
- (3) 提取 Neighbor Advertisement 消息中的目标地址, 目标地址可以是单播地址或任意播地址, 但是不可以是多播地址。如果 Flag 中的请求标志设置为 1, 并且报文的目的地址是多播地址, 则判定为无效报文。
- (4) 更新 Neighbor Advertisement 消息, 提取目标链路选项。目标链路选项的格式与源链路选项格式相同, 但是其类型值为 2。其即是地址解析最终结果。
- (5) 将之前步骤所有的结果进行汇总, 根据请求标志和覆盖标志, 决定是否将物理地址加入缓存。同时, 由于在发送 Neighbor Solicitation 消息的时候, 已经添加过一次不完全的邻居发现缓存表目, 因此根据 IPv6 地址查询不到对应的缓存条目时, 应放弃更新缓存。

转发平台发送一个 Neighbor Advertisement 消息时, IPv6 报文处理模块要根据之前的 Neighbor Solicitation 消息的相关信息做处理。其处理流程如下:

- (1) 在 Neighbor Solicitation 消息的基础上, Neighbor Advertisement 消息的生成是通过对原有的 Neighbor Solicitation 消息进行修改而完成的。因为是利用现有的邻居发现协议报文, 不需要是由内存池对重新生成报文。
- (2) Neighbor Advertisement 消息的目的地址, 应该是原来 Neighbor Solicitation 消息中的源地址。但是如果原来的 Neighbor Solicitation 消息的源地址是未确定地址, 那么 Neighbor Advertisement 消息中的 IPv6 报头地址是所有节点的多播地址, 且请求标志为 0。源地址是本机地址。
- (3) 对于 Neighbor Advertisement 消息, ICMPv6 代码值为 136, 目标地址不变。如果 Neighbor Solicitation 消息含有源链路选项, 则 Neighbor Advertisement 消息要设置目的链路选项。如果 Neighbor Solicitation 消息是作为地址解析的请求报文, 则覆盖标志和请求标志均要设置为 1, 路由标志设置为 0。
- (4) 最后需要计算全部报文的校验和, 并将数据报文添加至发送队列。完成 Neighbor Advertisement 消息的发送。

由于在生成 Neighbor Solicitation 消息和 Neighbor Advertisement 消息的同时需要对 ICMPv6 报文进行填充, 其中包括类型值, 代码值和 ICMPv6 报文的校验和。ICMPv6 的校验和与 IPv4 时代的 TCP 协议, UDP 协议的算法相同。校验和算法的本质是对数据报文进行以 16 比特为单位的二进制反码求和。运输层协议在计算校验和的时候, 除了需要计算自身数据和上层数据报文的校验和的同

时，运输层的校验和计算还需要计算一个伪头部。对于 IPv6 报文而言，伪头部是指 IPv6 报头的目的地址，源地址，网络层协议的协议类型和数据报文大小。伪头的数据结构如下所示：

```
union ipv6_psd_header {
    struct {
        uint8_t src_addr[16]; /* IP address of source host. */
        uint8_t dst_addr[16]; /* IP address of destination host(s). */
        uint32_t len; /* L4 length. */
        uint32_t proto; /* L4 protocol - top 3 bytes must be zero */
    } __attribute__((packed));
    /* allow use as 16-bit values with safe aliasing */
    uint16_t u16_arr[0];
} psd_hdr;
```

IPv6 网络运输层的校验分为以下几个步骤：首先，获得运输层协议和上层载荷数据的总长度，计算运输层协议和上层载荷数据的以 16 比特为单位的二进制反码和。其次计算 IPv6 伪头的 16 比特为单位的二进制反码求和。如果最后求的校验和的高 16 比特不为 0，则将校验和的高 16 比特和低 16 比特相加，直到最终结果的高 16 比特为 0 位置。最后校验和的结果是一个 16 比特的值

上面介绍了 IP 转发平台的所有相关设计和实现。无论是在 DPDK 平台上实现任何功能，基本的 IP 数据报文的转发都是必不可少的。由于 DPDK 开发环境本身并没有一个免费的开源协议栈。因此，结合 IPv4/IPv6 地址翻译器的实现目标，本课题实现了一个通用的 IP 转发平台。考虑到后续应用的二次开发，IP 转发平台具有良好的代码设计和接口设计。通过 C 语言的关键字实现了相关代码的隐藏，程序具有良好的扩展性和可移植性。开发人员无需担心已完成功能对新组建的影响。并且通过一个集中的数据模块，使得后续开发可以更加的规范和有效。

4.2 有状态的 IPv4/IPv6 地址翻译器的设计

有状态协议地址翻译技术 NAT64 方案由标准文档 RFC6146 定义，NAT64 方案可以实现从 IPv6 网络到 IPv4 网络通信的过渡场景。NAT64 有状态地址翻译技术主要基于 IP 地址映射技术，端口重用技术，其可以实现 N:1 的 IPv6 地址映射到 IPv4 地址的映射。

4.2.1 IPv4/IPv6 地址翻译器与 IP 转发平台的结合

由于 IPv4/IPv6 转发平台预留了相关的接口，NAT64 可以使用 IP 转发平台的转发函数，对转换完成的 IP 数据报进行转发。由于 NAT64 中的相关数据库依旧需要使用系统内存，因此，我们在 NAT64 方案中依旧使用了 DPDK 工具作为开发平台，在 NAT64 规范中定义的绑定信息表，Session 表等使用了 DPDK 平台开发工具中的内存接口，地址的匹配则使用了最长前缀匹配接口。并且 NAT64 所支持的相关报文格式，在 IP 转发平台中均有涉及。地址翻译器甚至可以利用原有的转发平台中的相关数据结构，以减少代码的编写量。完整的 IPv4/IPv6 地址翻译器的架构图如图 4-8 所示。

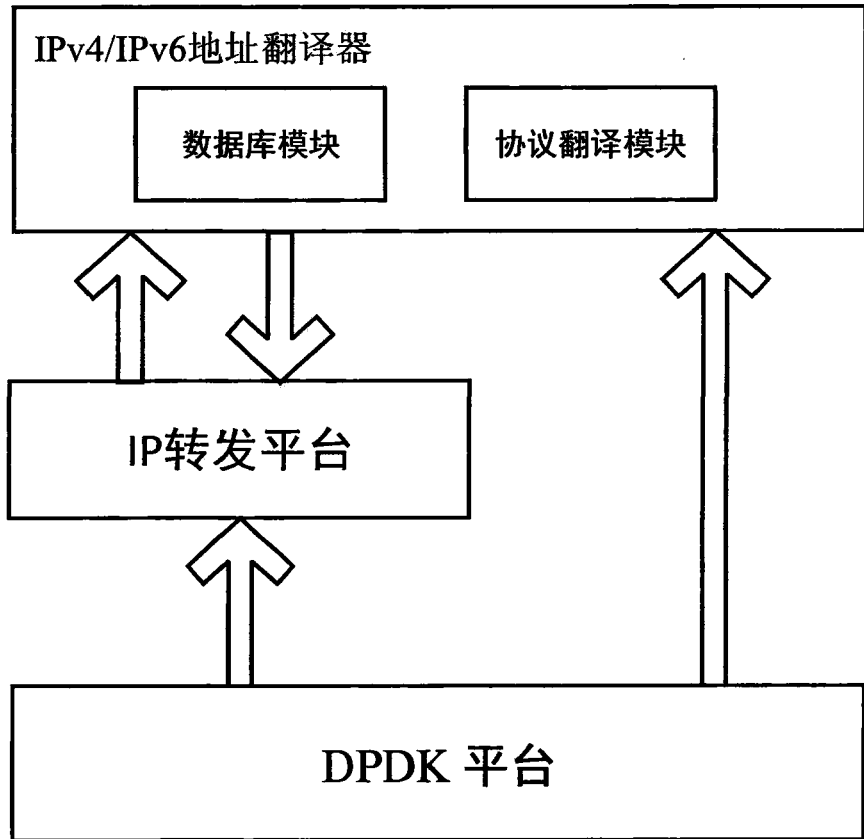


图 4-8 IPv4/IPv6 地址翻译器的整体架构图

4.2.2 数据库模块

在 IPv4/IPv6 地址翻译器中有 2 个重要的数据库，其分别是信息绑定表和 Session 表。2 张数据缓存表在 IPv4/IPv6 地址翻译器中的作用在前文中已经介绍过。信息绑定表和 Session 表的数据结构如下所示。

```
struct bib_entry {
    struct ipv4_transport_addr ipv4_addr;
    struct ipv6_transport_addr ipv6_addr;
    int16_t static;
```



```

rte_spinlock_t lock;
uint32_t refcounter;
struct rte_timer timer;
/** Appends this entry to the database's IPv6 index. */
struct rb_node tree6_hook;
/** Appends this entry to the database's IPv4 index. */
struct rb_node tree4_hook;
}__rte_cache_aligned;
struct session_entry{
    struct ipv6_transport_addr remote6;
    struct ipv6_transport_addr local6;
    struct ipv4_transport_addr local4;
    struct ipv4_transport_addr remote4;
    struct bib_entry * bib_entry;
    unsigned long update_time;
    rte_spinlock_t lock;
    uint32_t refcounter;
    uint8_t state;
    struct rb_node tree6_hook;
    struct rb_node tree4_hook;
}__rte_cache_aligned;

```

bib_entry 数据结构是信息绑定表的数据结构。其中主要记录了地址翻译的地址绑定信息，包括一对 IPv4 地址和 IPv6 地址，计时器和红黑树的索引值。**session_entry** 数据结构表示了 Session 表中的表项内容，Session 表是记录有状态会话中涉及到的相关信息。**session_entry** 数据结构中记录了 IPv4 的源地址和目的地址，IPv6 的源地址和目的地址等字段。对于有状态的网络层为 IPv6 协议，运输层为 TCP 协议报文，地址翻译器会从 IPv6 报头中提取源地址和端口号，存放到信息绑定表中。同时地址翻译器随机选择一个 IPv4 地址作为翻译后 IPv4 报头的源地址，同时分配一个新的端口号给这个数据包。然后地址翻译器将相关的数据放入 Session 表。

由于绑定信息表和 Session 缓存的空间大小是随着时间的推移而不断变化，而且我们并不知道 IPv6 网络中的主机数量，所以我们不能预先预估出绑定信息表和 Session 缓存的空间的上限值。我们不能使用类似于哈希的数据结构去实现绑定信息表和 Session 缓存。在综合考察了现有的信息绑定表和 Session 表的使

用频率，表项大小，增长程度等指标的情况下，本课题使用 Linux 内核中的红黑树数据结构实现绑定信息表和 Session 缓存的查找和存储。Linux 内核中所实现的红黑树本质上是一个二叉搜索树。搜索树在插入过程中按照键值大小进行插入，由于红黑树在一般的二叉搜索树的数据结构中加入了颜色的区别，并且定义了一些列规则。红黑树的性质决定了其在增删改查等操作的效率很高，适合于数据库的搭建。

4.2.3 协议翻译块

IPv4/IPv6 地址翻译器的数据包翻译模块只支持从 IPv6 网络用户发起请求，不支持从 IPv4 网络用户发起用户请求。同时，如前所述，地址翻译器还需要在内存中维护相关的数据库缓存，所有的翻译信息都存储在数据库缓存中。

有状态协议地址翻译器的数据包翻译模块工作流程如图 4-9 所示。

当 IPv4/IPv6 地址翻译器接收到一个 IPv6 数据报文后，IP 转发平台对 IPv6 报文进行解包，找出相对应的源地址，目的地址和端口号。翻译模块向信息绑定表查询，检查信息绑定表中是否已经存在对应的映射项目。如果不存在，翻译模块向绑定表中添加一个映射项目。同时，翻译模块根据映射条目对数据报文进行翻译。随机选择一个 IPv4 地址作为源地址，并将 IPv6 目的地址转换为可识别的目的 IPv4 地址，生成一个可由 IP 转发平台转发的 IPv4 数据报文。如果是运输层协议是 TCP 或 UDP 协议，则翻译模块还要进行端口号的替换，以达到端口复用的目的。

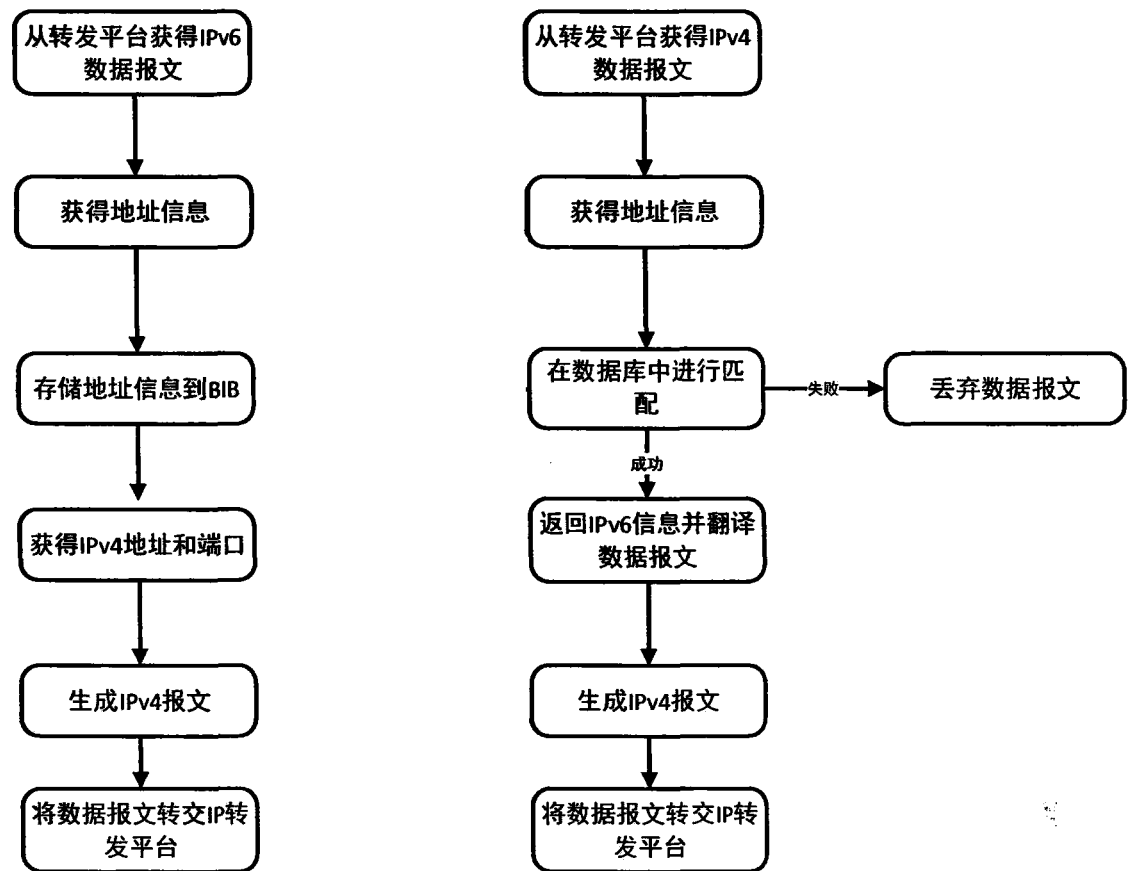


图 4-9 翻译模块工作流程

当 IPv4/IPv6 地址翻译器收到一个 IPv4 数据报文，转发平台对 IPv4 数据包进行解包，得到数据报文中的 IPv4 目的地址和传输层协议的端口号码。地址翻译模块从 IP 转发平台获得对应的信息，并通过数据库进行查询，检查此数据报文之前在数据库中的缓存项目。由于 NAT64 方案只支持从 IPv6 网络发起请求到 IPv4 网络，因此，当 IP 转发平台接收到一个 IPv4 数据报文的时候，这个 IPv4 数据报文一定已经在翻译器的数据库中记录过。因此，当数据库找不到此报文的相关信息的时候，翻译器丢弃报文。如果数据库中存在相对应的记录，翻译模块根据信息绑定表和 Session 表中的信息对地址进行转换，根据信息绑定表中的 IPv6 地址，转换数据报文的地址。同时，翻译模块把源 IPv4 地址转换成 IPv6 地址，如果上层协议是 TCP 或 UDP，翻译模块还要负责转换对应的端口号，从而生产一个 IPv6 数据报文。

第五章 IPv4/IPv6 地址翻译器的部署与测试

5.1 测试目的

对 IPv4/IPv6 地址翻译器的测试目的是验证 IP 转发平台与其他标准的协议栈之间的互通性和所实现的翻译器的可用性。验证本方案的可行性。

关于 IPv4/IPv6 地址翻译器的测试主要分两个方面，分别是互通性测试和业务验证测试。互通性测试主要包括搭建 IPv6 过渡技术的网络拓扑，先进行简单的端到端互通性测试，再做业务测试，业务验证测试包括在互通性测试的基础之上验证 NAT64 翻译技术对数据报文翻译的支持程度。

5.2 环境搭建和测试流程

无论是 IP 转发平台还是地址翻译器均要在 DPDK 的环境中运行。因此，测试环境搭建主要是 DPDK 环境的编译和运行。同时，由于 DPDK 需要绑定相关的网卡驱动，通过使用自己的网卡驱动达到接管系统网卡的目的。DPDK 环境的编译，安装，系统网卡的绑定主要依靠 DPDK 工具自带的脚本实现。当网卡被 DPDK 平台接管后，操作系统是无法发现网卡存在的。

整个测试流程如图 5-1 所示。步骤如下：

- (1) 搭建测试环境：测试环境的概念包括软硬设备的搭建、网络的搭建和基本配置。
- (2) 互通性测试，若互通性测试未通过，进行故障的排查，查找问题故障出现的所在，如果故障被发现顺利解决，再次重复互通性测试，直到互通性测试通过，记录测试结果。
- (3) 业务验证测试，如果业务验证测试未通过，进行故障的排查，查找问题故障出现的所在，如果故障被发现顺利解决，再次重复业务验证，直到测试通过，记录测试结果，结束测试。

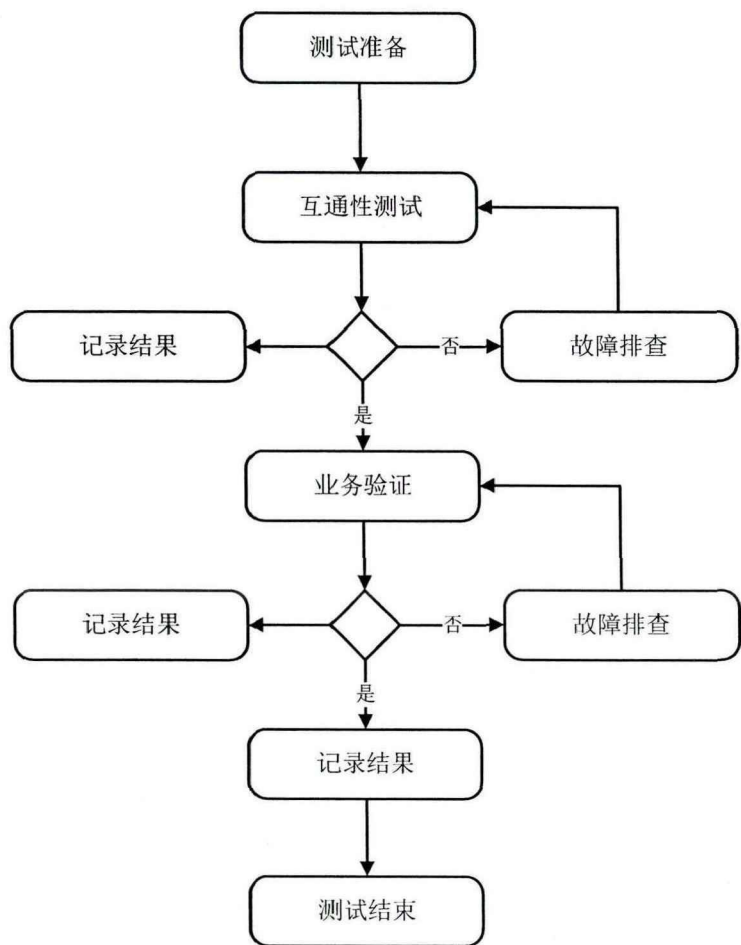


图 5-1 测试流程

5.3 测试拓扑和测试项目

测试拓扑如图 5-2 所示。

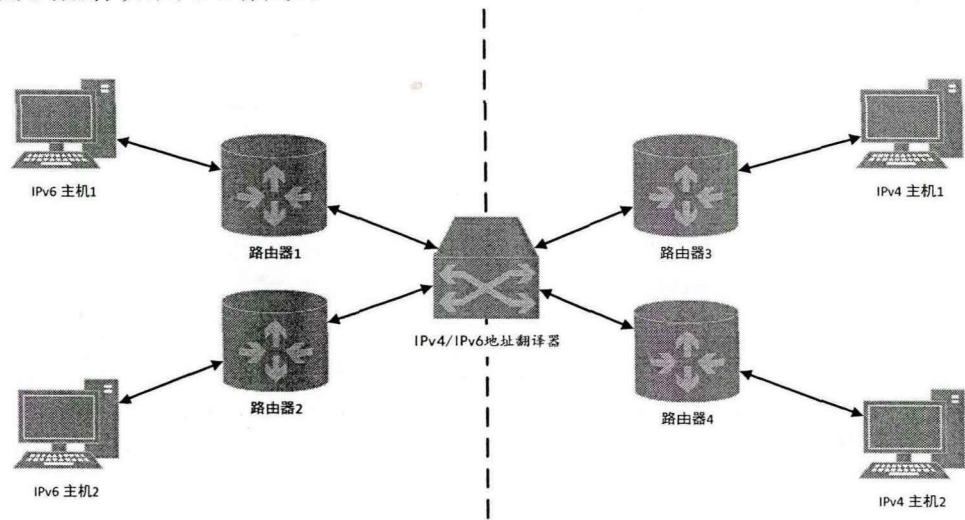


图 5-2 测试拓扑

5.3.1 连通性测试

连通性测试项目如表 5-1 到 5-2 所示，其显示了 IPv4 数据报文与 IPv6 数据保报文的测试结果。

表5-1 IPv4连通性测试

测试项目	IP 转发平台的 IPv4 连通性测试	
测试目的	测试 IPv4 到 IPv4 报文的基本连通性实现	
预置条件	设置 IPv4 主机，IPv4 路由器以及对应路由指向到地址翻译设备	
测试步骤		预期结果
1.配置翻译器的 IP 转发模块，包括 IPv4 地址，IPv4 路由和系统配置		成功配置符合要求的条件参数
2.进行终端 A 到终端 B 的 ping 测试		测试使用的 ping 包经过转发模块后不可正确返回
3.启动翻译器的 IP 转发模块，进行终端 A 到终端 B 的 ping 测试		测试使用的 ping 包经过转发模块后可正确处理并返回
测试结果	通过	

表5-2 IPv6连通性测试

测试项目	IP 转发平台的 IPv6 连通性测试	
测试目的	测试 IPv6 到 IPv6 报文的基本连通性实现	
预置条件	设置 IPv6 主机，IPv6 路由器以及对应路由指向到地址翻译设备	
测试步骤		预期结果
1.配置翻译器的 IP 转发模块，包括 IPv6 地址，IPv6 路由和系统配置		成功配置符合要求的条件参数
2.进行终端 A 到终端 B 的 ping6 测试		测试使用的 ping6 包经过转发模块后不可正确返回
3.启动翻译器的 IP 转发模块，进行终端 A 到终端 B 的 ping6 测试		测试使用的 ping6 包经过转发模块后可正确处理并返回
测试结果	通过	

5.3.2 业务验证测试

业务验证测试是关于 IPv4/IPv6 地址翻译器功能的验证。表 5-3 显示了 IPv4 数据报文与 IPv6 数据保报文的测试结果。

表5-3 IPv4/IPv6地址翻译器的功能验证

测试项目	IPv4/IPv6 地址翻译器的功能验证	
测试目的	测试 IPv6 报文到 IPv4 报文翻译的基本连通性实现	
预置条件	设置 IPv6 主机，IPv4 主机，路由器以及 IPv4/IPv6 地址翻译器设备	
测试步骤		预期结果
1.配置翻译器的 IP 转发模块和地址翻译模块。		成功配置符合要求的条件参数
2.进行终端 A 到终端 B 的 ping6 测试和发包测试		测试使用的 ping6 和 TCP 数据包经过地址翻译器设备后不可正确返回。
3.启动翻译器，进行终端 A 到终端 B 的 ping6 测试和发包测试		测试使用的 ping6 和 TCP 数据包经过地址翻译器设备后可以正确返回并处理报文。
测试结果	通过	

第六章 总结与展望

6.1 全文总结

伴随着互联网的蓬勃发展, IPv4 网络在获得极大成功的同时, 其也曝露出一些缺陷, 尽管后续通过 NAT, CIDR 等技术的不断修补, 但是面对如今的全球互联网, 原有的 IPv4 网络已经不能满足, IPv4 协议必须要被新的, 更好的协议所取代。在经过二十几年的努力, IPv6 协议作为下一代的 IP 网络协议的标准版本, 解决了 IPv4 协议的诸多问题。首先, IPv6 解决了 IPv4 地址的短缺问题, IPv6 地址从原来 IPv4 地址的 32 位地址长度扩展成为 128 位, 使得世界上的每一台设备都可以有属于自己的 IP 地址。同时, IPv6 协议在摒弃了 IPv4 诸多缺点的情况下, 也继承了原有 IPv4 协议的优点。但是由于 IPv4 协议与 IPv6 协议并不兼容, 因此在现在大流量数据并发的背景下, 本体研究了一种高性能的 IPv4/IPv6 地址翻译器。

本课题在研究 IPv4/IPv6 地址翻译技术的前提下, 重点研究了有状态 IPv4/IPv6 地址翻译技术, 根据 RFC 标准实现了一个有状态的 IPv4/IPv6 地址翻译器, 部署在 IPv6 网络与 IPv4 网络之间, 实现 N:1 的 IPv6 地址与 IPv4 地址的转换, 达到了 IPv4 到 IPv6 过渡的目的。同时, 本文还对 TCP/IP 网络协议栈和高性能的数据处理方式做了深入的学习, 通过学习底层的数据报文处理知识, 构建了一个高性能的 IP 数据报文处理平台。通过高性能的数据处理平台与有状态 IPv4/IPv6 协议地址翻译器的结合, 我们可以进一步提高现有 IPv4/IPv6 协议地址翻译器的翻译速度, 加快数据报文的处理和转发。

本文主要完成了如下几方面的工作:

- 一、学习研究 IPv6 网络和 IPv6 过渡技术中的相关概念与技术, 着重研究了有状态的地址转换过渡技术;
- 二、详细地学习高性能平台 DPDK 的相关技术, 对如果在操作系统层面和应用软件层面处理高并发性的请求;
- 三、详细地学习了 IPv4 数据报文和 IPv6 数据报文的收发原理和地址解析过程。参照现有的 Linux 网络协议栈, 利用高性能平台实现了 IPv4 和 IPv6 数据报文的转发。

四、深入学习了 RFC6146, 参考现有的标准, 结合 DPDK 平台, 实现了一个有状态的 IPv4/IPv6 协议地址翻译器。

6.2 工作展望

本课题的主要目的是设计实现一个高性能的有状态的 IPv4/IPv6 协议地址翻译器。IPv4/IPv6 协议地址翻译器是搭建在 DPDK 平台之上的。本课题现阶段已基本完成了所涉及到的需要实现的内容, 翻译器可以保证正常的翻译流程。凡是在实际的运行环境中, 仍旧有以下问题。

- (1) 底层 IP 转发平台的路由设计还不完善, 目前只能通过对路由表项进行静态配置, 转发平台还不具备学习路由的能力。后续的实现需要是实现独立的路由系统, 使得转发平台可以动态的对路由进行修改。
- (2) 邻居发现协议仅仅实现了 Neighbor Solicitation 消息和 Neighbor Advertisement 消息, 在邻居发现协议中的其他协议并没有实现。后续将邻居发现协议完全实现, 包括 Router Solicitation 消息, Router Advertisement 消息和 Redirect 消息。
- (3) 进一步通过软件的方式对整个系翻译器进行重构, 加快数据报文的处理速度, 完善电源管理。并使得程序可以用于虚拟化的平台。
- (4) 针对移动用户, 将 MIPv6 的处理添加进入协议栈, 使得地址翻译器能够支持移动设备。

参考文献

- [1] 白莹,赵振东,戚银城,李洁 从 IPv4 到 IPv6 的改进和过渡 电力系统通信 2004 年第 9 期. 40 - 43
- [2] 王晓峰,吴建平,崔勇 互联网 IPv6 过渡技术综述 小型微型计算机系统 2006 年 03 期 385-389
- [3] 杨国良, 李阳春, 伍佑明 IPv6 技术、部署与业务应用 人民邮电出版社 2011 5-8
- [4] S. Deering,R. Hinden Internet Protocol Version 6 (IPv6) Specification RFC2460 Network Working Group 1998
- [5] The C10K problem <http://www.kegel.com/c10k.html>
- [6] C10M <http://c10m.robertgraham.com/p/blog-page.html>
- [7] 马严 赵晓宇 IPv4向IPv6过渡技术综述 北京邮电大学学报 第25卷第4期 2002年12月 1-5
- [8] W. Richard Stevens, Bill Fenner, Andrew M. Rudoff Unix Network Programming, Volume 1: The Sockets Networking API 3rd Edition Addison-Wesley Professional 2003 209-214
- [9] Ulrich Drepper The Need for Asynchronous, Zero-Copy Network I/O Ottawa Linux Symposium. 2006: 247-260.
- [10] Data Plane Development Kit <http://www.dpdk.org>
- [11] Silvia Hagen IPv6 Essentials O'Reilly Media, Inc, 2006 1-16
- [12] David C. Plummer An Ethernet Address Resolution Protocol RFC 826 Network Working Group 1982
- [13] T. Narten E. Nordmark W. Simpson H. Soliman Neighbor Discovery for IP version 6 (IPv6) RFC4861 Network Working Group
- [14] A. Conta S. Deering M. Gupta, Ed. Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification RFC 4443 Network Working Group 2006
- [15] Lamport, L How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs Computers, IEEE Transactions on 1979
- [16] Maurice Herlihy Nir Shavit The Art of Multiprocessor Programming, Revised Reprint Morgan Kaufmann 2012 11-90
- [17] Programmer's Guide <http://dpdk.org/doc> 2014
- [18] Jake Edge A lockless ring-buffer <http://lwn.net/Articles/340400/>
- [19] M. Bagnulo P. Matthews I. van Beijnum Stateful NAT64: Network Address and Protocol Translation from IPv6 Clients to IPv4 Servers RFC6146 Internet Engineering Task Force 2011
- [20] C. Bao C. Huitema M. Bagnulo M. Boucadair X. Li IPv6 Addressing of IPv4/IPv6 Translators RFC 6052 Internet Engineering Task Force (IETF) 2010

[21] M. Bagnulo A. Sullivan P. Matthews I. van Beijnum DNS64: DNS Extensions for Network Address Translation from IPv6 Clients to IPv4 Servers RFC 6147 Internet Engineering Task Force (IETF) 2011

致 谢

白驹过隙,时光飞逝,两年多时光很快就要过去。在北京邮电大学攻读硕士研究生的这段时光也即将结束。我马上要从一个青涩的学生变成一个走向成熟。在北京邮电大学网络技术研究院信息网络中心工作和生活的日子里,我受益良多。在这里,我不仅仅接触到了世界上最新的技术和最前沿的知识,还开阔了自己的眼界,极大的丰富了自己。从入学至今,我一直都接受着实验室的老师们和同学们的帮助,没有他们的指导和给予我的帮助,我不可能有现在这样的水平,在这里请允许我向他们表达我的感激之情。

我首先想向我的导师马严老师表达感谢,马严老师严谨的治学态度,丰富的理论和实践知识和敬业的精神,一直影响着我,他是我学习和工作的榜样。其次我还要感谢我实验室项目指导老师赵钦老师,正是赵钦老师无微不至的教导,工作一丝不苟的态度,办事尽善尽美的工作要求感染了我。每当我在实验室项目中遇到困难的时候,赵老师总是耐心的帮我解决相关的文通,给我启发和指导给予了我上遇到困难之时,总是耐心的给予我启发和指导。我还要感谢闫岫老师,黄小红老师,吴军老师,柯宏力老师、王志谦老师在这三年中对我的细心指导。

同时我还要感谢实验室的师兄师姐师弟师妹们,与你们一起工作是我能够更清楚的认识自己。特别是彭飞翔、林潇俊、连捷、张洁、高静雯、高志腾等,感谢他们在项目完成和论文的撰写上给予我的支持。

感谢我的舍友:徐扬,王超,郑楠,赵晓森,陈一鸣。是他们在这个三年来跟我生活在一起,与我一起进步。最后还要感谢我的父母和家人,谢谢你们一直对我的关心和支持。

衷心感谢各位论文评审的专家,领导和老师为本文提出宝贵修改建议。

硕士期间论文发表

[1] 张宇烜, 闫岫, 徐希炜 非对称的 IPv6 地址翻译技术的实现与分析 《通信学报》第 35 卷第 Z1 期 2014 年 10 月 P141-145